

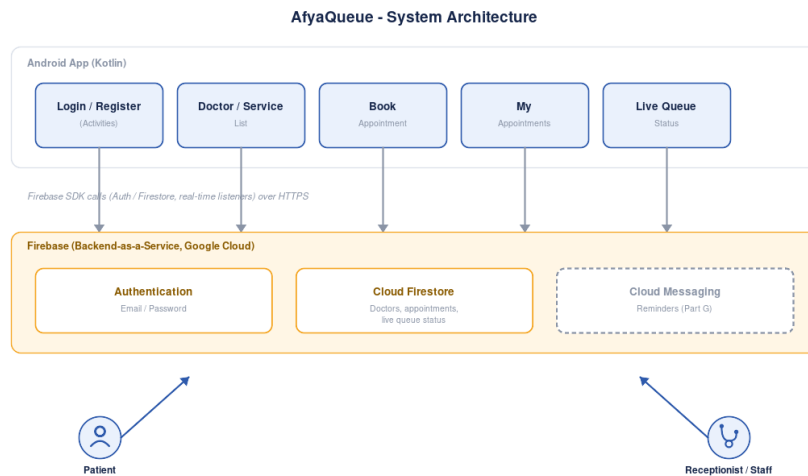
MOBILE APPLICATION DEVELOPMENT

A Step-by-Step Teaching Manual

From installing Android Studio to publishing on Google Play

Building AfyaQueue

A Healthcare Appointment & Queue Management App for Android



Course: Mobile Application Development (CSC 417)

Moi University - Department of Computer Science

*Project concept: AfyaQueue - Healthcare Appointment & Queue Management App, proposed by
Marion Kibet (COM/5916/24)*

Prepared as a teaching resource for classroom demonstration and student practice

Table of Contents

Right-click the list below in Microsoft Word and choose “Update Field” (or press F9) to generate page numbers after opening this document.

Table of Contents.....	2
Introduction	5
About this manual	5
Learning objectives	5
What you will build	6
Tools and technologies used.....	6
Prerequisites.....	7
How this manual is organised	7
Part A — Setting Up the Development Environment.....	8
A.1 System requirements.....	8
A.2 Step 1 — Download Android Studio.....	8
A.3 Step 2 — Run the installer	9
A.4 Step 3 — Create a Virtual Device (emulator).....	10
A.5 A short tour of Android Studio.....	11
A.6 Verifying your installation.....	12
Part B — Setting Up the Firebase Backend	13
B.1 Step 1 — Create a Firebase project.....	13
B.2 Step 2 — Register the Android app	13
B.3 Step 3 — Add the Firebase SDKs to your project	14
B.4 Step 4 — Enable Email/Password Authentication	16
B.5 Step 5 — Create the Firestore Database.....	16
B.6 Securing your data before launch	17
Part C — Planning AfyaQueue.....	19
C.1 The problem, recapped	19
C.2 System architecture	19
C.3 Screens and navigation flow	20
C.4 Data model	22
C.5 How the live queue actually works	22
C.6 Scope for this manual	23

Part D — Building AfyaQueue, Screen by Screen.....	25
D.0 Create the Android Studio project.....	25
D.1 App icon and branding	25
D.2 Data models	26
D.3 Splash, Login and Register.....	29
D.4 Home (Doctor / Service List).....	34
D.5 Doctor Detail / Book Appointment	37
D.6 My Appointments.....	42
D.7 Live Queue Status — the heart of AfyaQueue	45
D.8 Staff Home — Today's Queue	48
D.9 Add Doctor / Service (staff role).....	52
D.10 Wiring it all together	53
Part E — Testing AfyaQueue.....	55
E.1 Running on the emulator	55
E.2 Running on a physical device	55
E.3 Testing the real-time queue — the most important test in this manual	56
E.4 Test checklist.....	56
E.5 Debugging with Logcat.....	58
E.6 Common errors and fixes	58
Part F — Publishing to the Google Play Store	60
F.1 Pre-launch checklist	60
F.2 Generating a signed app bundle	60
F.3 Creating a Google Play Console account	61
F.4 Creating the app and the store listing	61
F.5 Content rating, target audience and data safety.....	62
F.6 Uploading the release.....	62
F.7 Review and publish timeline.....	64
Part G — Extending AfyaQueue (Stretch Goals)	65
G.1 Push Notifications	65
G.2 Facility Announcements Board.....	65
G.3 Further improvement ideas.....	66
Appendix.....	67

A. Complete Firestore security rules	67
B. Troubleshooting quick reference	68
C. Glossary	69
D. Suggested assessment rubric.....	70
E. References and further reading	71

Introduction

About this manual

This manual is a complete, classroom-ready walkthrough for the Mobile Application Development course. It takes a student from an empty computer with no development tools installed, all the way through designing, coding, testing and publishing a real Android application.

The application used throughout this manual, AfyaQueue, is built from an actual student project proposal: a mobile app that lets patients book appointments at a hospital or clinic, receive a digital queue number, and track their position in the queue in real time. Using a genuine, previously-submitted project idea (rather than a generic demo app) gives students a realistic sense of how a class project brief becomes working software.

Project source:

The AfyaQueue concept, problem statement and feature list used in this manual are drawn from the project proposal “AfyaQueue – Healthcare Appointment & Queue Management App” submitted by Marion Kibet (COM/5916/24) for CSC 417, addressing long waiting times and poor appointment coordination at healthcare facilities.

Learning objectives

By the end of working through this manual, a student should be able to:

- Install and configure Android Studio and the Android SDK on a Windows, macOS or Linux computer.
- Create and run an Android app on both an emulator and a physical device.
- Explain the role of a backend-as-a-service (Firebase) and connect an Android app to Authentication and Firestore.
- Design a simple, coherent app architecture and navigation flow before writing code.
- Build multiple connected screens (activities) in Kotlin using XML layouts, RecyclerView lists, and Material Design components.
- Implement a real-time data feed using Firestore snapshot listeners, so one user's action (a receptionist calling the next patient) updates another user's screen (a patient's queue status) instantly.
- Test an app systematically and use Logcat to diagnose problems.
- Prepare, sign and publish an Android app on the Google Play Store.

What you will build

Following this manual end-to-end produces a working core version of AfyaQueue with these features:

- Registration and login for patients and clinic staff (Firebase Authentication).
- A browsable list of doctors and services with their availability (Cloud Firestore).
- Appointment booking by date and time slot, with an automatically assigned digital queue number.
- A live queue status screen showing the patient's number, the number currently being served, and an estimated wait — updating in real time.
- A My Appointments screen listing upcoming and past bookings, with cancel/reschedule actions.
- A staff-facing screen showing today's queue with a Call Next Patient action, and a screen to add new doctors/services.

Push notifications and hospital/clinic announcements, which appear in the original project proposal, are covered in Part G as extension exercises once the core app works — they follow the same patterns taught in Part D, and are a good graded extension task for stronger students.

Tools and technologies used

Purpose	Tool / Technology
IDE	Android Studio (Meerkat or newer)
Language	Kotlin
UI toolkit	Android Views + XML layouts, Material Components
Backend	Firebase (Authentication, Cloud Firestore)
Real-time updates	Firestore snapshot listeners
Notifications (extension)	Firebase Cloud Messaging
Version control (recommended)	Git and GitHub
Distribution	Google Play Console

Prerequisites

Students should ideally have:

- Basic programming experience (variables, functions, classes, conditionals, loops) — prior exposure to Java or Kotlin helps but is not mandatory.
- A computer with at least 8 GB of RAM (16 GB strongly recommended) and about 10 GB of free disk space.
- A reliable internet connection — the SDK, Firebase and Play Console steps all require it.
- A free Google account, used for both Firebase and Google Play Console.

How this manual is organised

Part	What it covers
A	Installing Android Studio and setting up the SDK and emulator
B	Creating and configuring the Firebase backend
C	Planning AfyaQueue: architecture, screens and data model
D	Building every screen of the app, step by step, with full code
E	Testing the app on an emulator and a real device
F	Publishing the app to the Google Play Store
G	Extension exercises: push notifications, announcements

Instructor's note: each part is self-contained enough to be taught as a separate lesson or lab session. A suggested pacing is one part per 2–3 hour lab session, giving a seven-session module.

Part A — Setting Up the Development Environment

Every Android app starts with the same foundation: a working copy of Android Studio, the official IDE, plus the Android SDK and an emulator to test on. This part walks through that installation from scratch, in the order a first-time student should follow it.

A.1 System requirements

Requirement	Minimum	Recommended
Operating system	Windows 10 (64-bit), macOS 12, or 64-bit Linux	Windows 11 / macOS 14+
RAM	8 GB	16 GB
Disk space	8 GB free	20+ GB free (SDK + emulator images grow quickly)
Screen resolution	1280 x 800	1920 x 1080

A.2 Step 1 — Download Android Studio

1. Open a web browser and go to **developer.android.com/studio**.
2. Click Download Android Studio and accept the terms and conditions.
3. Wait for the installer (about 1 GB) to finish downloading.

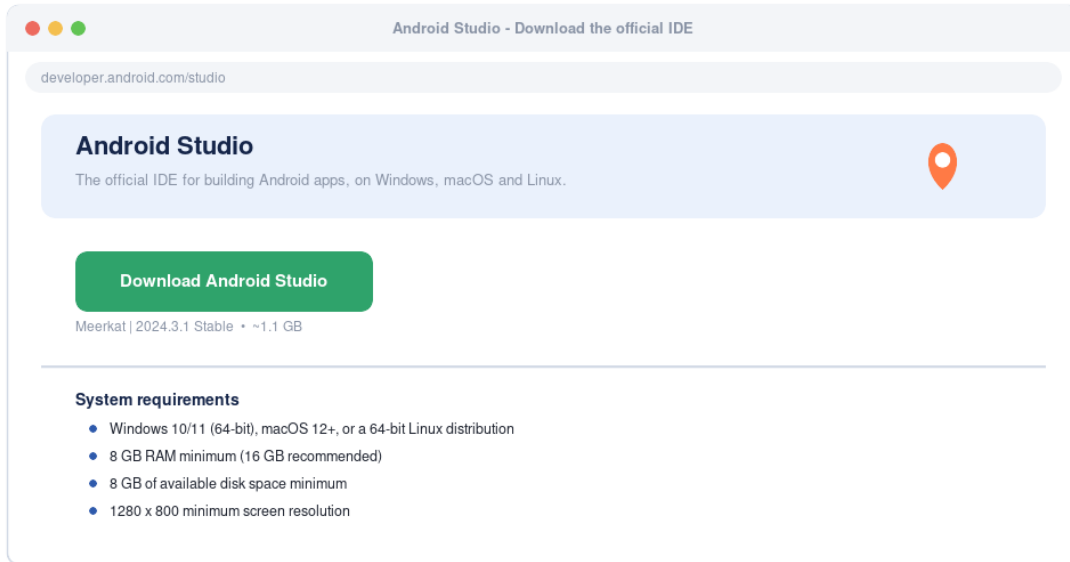


Figure A.1 — The Android Studio download page

A.3 Step 2 — Run the installer

Launch the file you downloaded and follow the setup wizard:

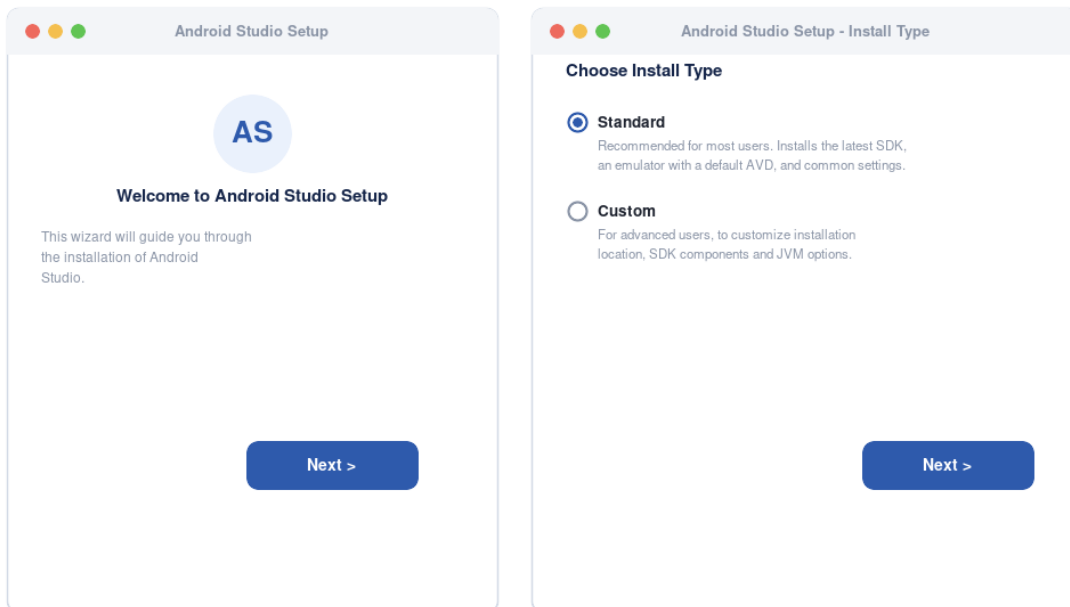


Figure A.2 — The Setup Wizard: welcome screen and install type

- Welcome screen: click Next.
- Install type: choose Standard — this installs the SDK, an emulator and sensible default settings. Choose Custom only if you need a non-default install location.

- UI theme: pick whichever you prefer (Light or Darcula); it has no effect on your app.

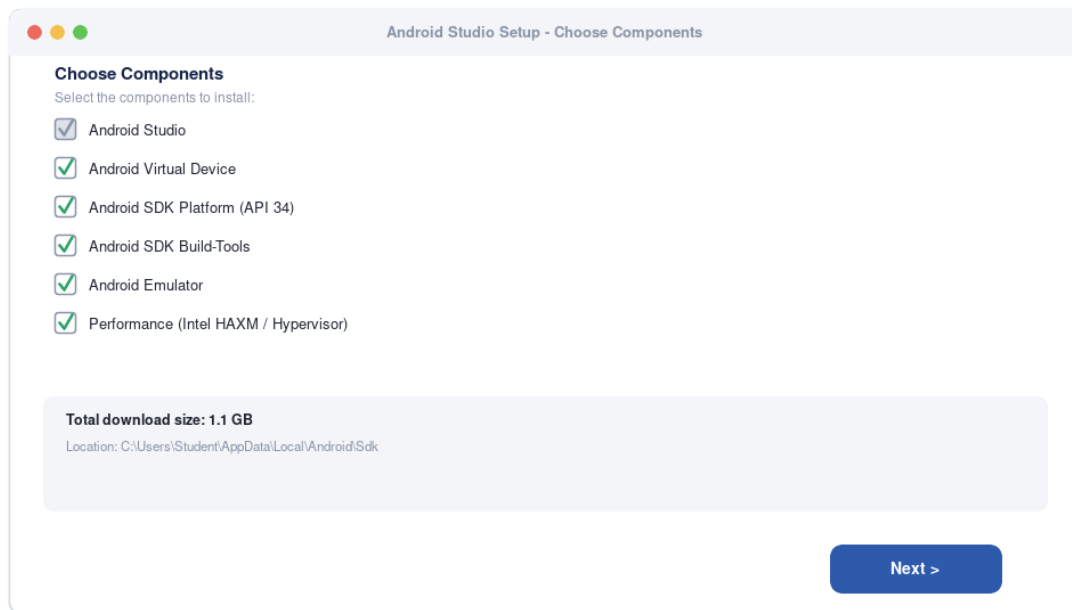


Figure A.3 — Confirming the SDK components that will be installed

Review the components list, confirm the install location has enough free space, and click Next, then Finish. The first launch downloads the SDK components, which can take 10–30 minutes depending on your connection — this is a good moment for a short break.

Common issue:

If the installer reports a missing HAXM/Hypervisor driver on Windows, make sure virtualization (VT-x/AMD-V) is enabled in your computer's BIOS/UEFI settings, then re-run the SDK Manager component install.

A.4 Step 3 — Create a Virtual Device (emulator)

An Android Virtual Device (AVD) lets you run and test your app on a simulated phone without needing a physical device.

1. From the Welcome screen, open More Actions → Virtual Device Manager (or Tools → Device Manager inside a project).
2. Click Create Device.
3. Select a phone profile — Pixel 7 is a good, modern default.
4. Choose a system image. Pick the newest API level with a Download link next to it (API 34, “UpsideDownCake”, is used throughout this manual), and click Download, then Next.

5. Click Finish. Your new emulator now appears in the Device Manager.

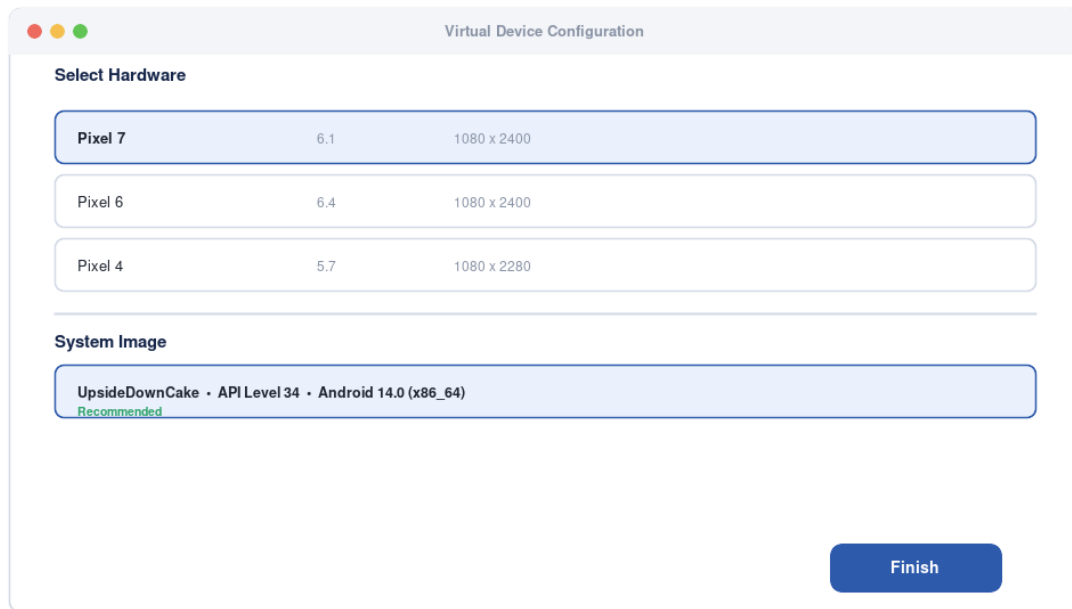


Figure A.4 — Creating a Pixel 7 virtual device running API 34

A.5 A short tour of Android Studio

Before writing any code, it helps to know where things live in the IDE window:

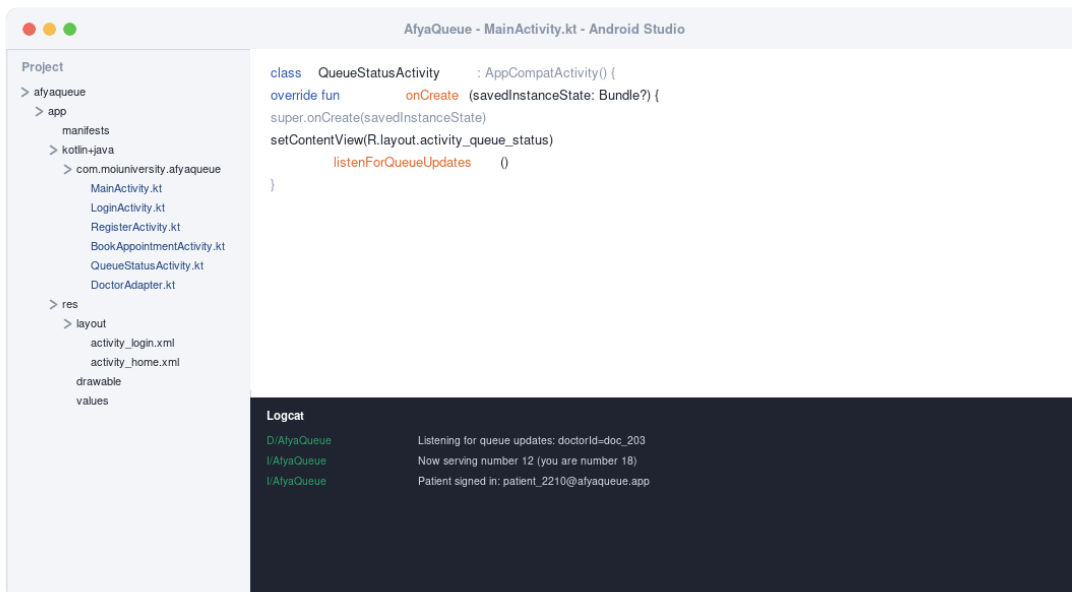


Figure A.5 — The Android Studio window: Project panel (left), code editor (top right), Logcat (bottom right)

Panel	What it's for
Project	Browse every file in your app: Kotlin/Java source, XML layouts, resources, manifest.
Editor	Where you write and edit code and layouts. Tabs open across the top.
Logcat	A live stream of log messages from your running app — essential for debugging (see Part E).
Run  button	Builds the app and launches it on the selected emulator or device.
Device dropdown	Choose which emulator or connected device Run will target.

A.6 Verifying your installation

Before moving on, confirm everything works end-to-end:

1. On the Welcome screen, click New Project.
2. Choose the Empty Views Activity template and click Next.
3. Set Name to VerifySetup, keep the default package name and location, set Minimum SDK to API 24 (Android 7.0), Language to Kotlin, and click Finish.
4. Wait for Gradle to finish syncing (progress bar at the bottom of the window).
5. Select your Pixel 7 emulator in the device dropdown and click the green Run  button.
6. The emulator boots (the first boot can take a few minutes) and shows “Hello World!” on screen.

Checkpoint:

If “Hello World!” appears on the emulator, your Android Studio installation, SDK and emulator are all working correctly and you are ready for Part B. If the build fails, re-open the SDK Manager (Tools → SDK Manager) and confirm the Android SDK Platform and Build-Tools shown in Figure A.3 are installed.

Part B — Setting Up the Firebase Backend

AfyaQueue needs somewhere to store user accounts, doctor/service information, appointments and the live queue status. Rather than building and hosting a custom server (a substantial project on its own), this manual uses Firebase, Google's backend-as-a-service platform. Firebase gives us two pieces for free, at student-project scale:

- Authentication — secure sign-up and login, without us writing any password-handling code ourselves.
- Cloud Firestore — a NoSQL, real-time database for doctors, appointments and the live queue status. Its real-time listeners are what let a patient's phone update the instant a receptionist calls the next patient.

B.1 Step 1 — Create a Firebase project

1. Go to **console.firebase.google.com** and sign in with a Google account.
2. Click Create a project (or Add project).
3. Name it AfyaQueue and click Continue.
4. Google Analytics is optional for this project — you can disable it.
5. Click Create project and wait about 30 seconds for it to be provisioned.

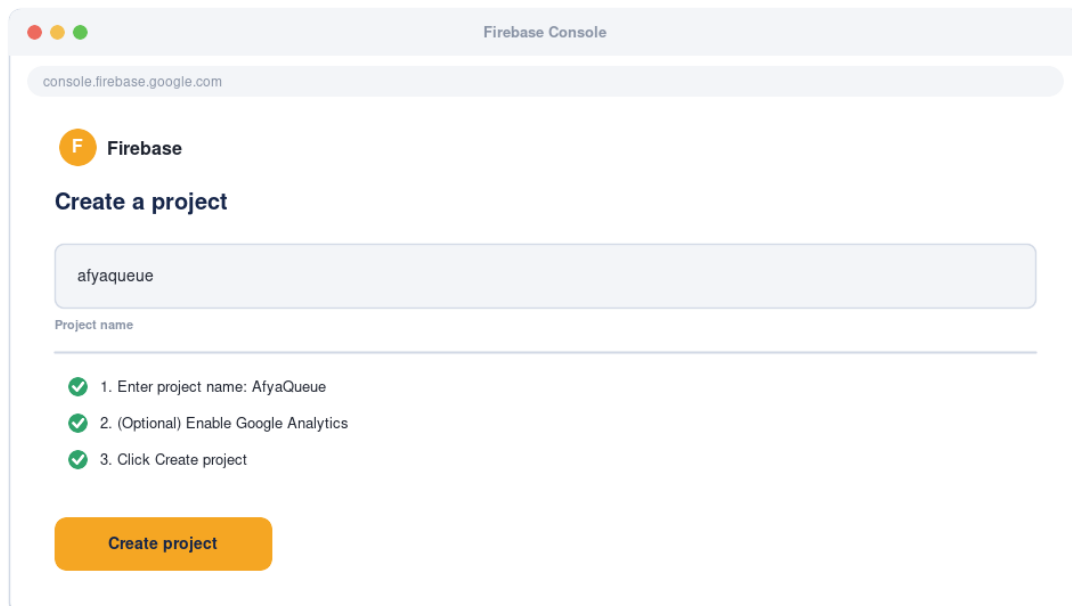


Figure B.1 — Creating the AfyaQueue project in the Firebase console

B.2 Step 2 — Register the Android app

Firebase needs to know your app's package name (its unique identifier) to connect the two.

1. In the Firebase console, click the Android icon to add an app.
2. Enter the Android package name exactly as you will use it in Android Studio, e.g. `com.moiuniversity.afyaqueue`.
3. (Optional) Enter an app nickname, e.g. “AfyaQueue”.
4. Click Register app.
5. Download the `google-services.json` file when prompted.

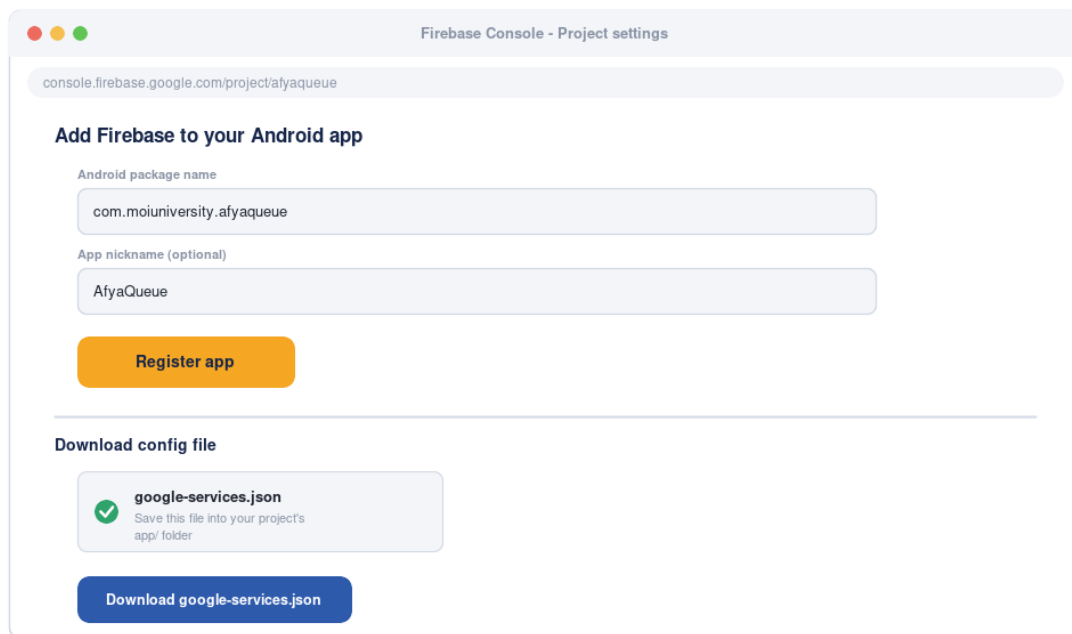


Figure B.2 — Registering the Android app and downloading google-services.json

Important:

The package name you register in Firebase must exactly match the `applicationId` in your Android Studio project's `build.gradle` file (Part D.0), or the app will fail to connect to Firebase.

B.3 Step 3 — Add the Firebase SDKs to your project

Back in Android Studio (once your project exists — see Part D.0):

1. Copy the downloaded `google-services.json` into your project's `app/` folder (the same folder as `build.gradle.kts` (Module :app)).
2. Open the project-level `build.gradle.kts` and add the Google Services plugin.

3. Open the app-level build.gradle.kts and apply the plugin, then add the Firebase dependencies.

build.gradle.kts (Project)

```
// Project-level build.gradle.kts
plugins {
    id("com.android.application") version "8.5.0" apply false
    id("org.jetbrains.kotlin.android") version "1.9.24" apply false
    id("com.google.gms.google-services") version "4.4.2" apply false
}
```

build.gradle.kts (Module :app)

```
// app-level build.gradle.kts
plugins {
    id("com.android.application")
    id("org.jetbrains.kotlin.android")
    id("com.google.gms.google-services")
}

dependencies {
    implementation("androidx.core:core-ktx:1.13.1")
    implementation("androidx.appcompat:appcompat:1.7.0")
    implementation("com.google.android.material:material:1.12.0")
    implementation("androidx.constraintlayout:constraintlayout:2.1.4")
    implementation("androidx.recyclerview:recyclerview:1.3.2")

    // Firebase (BoM keeps every Firebase library on a compatible version)
    implementation(platform("com.google.firebase:firebase-bom:33.1.2"))
    implementation("com.google.firebase:firebase-auth-ktx")
    implementation("com.google.firebase:firebase-firestore-ktx")

    // Firebase Cloud Messaging (used later, in Part G)
    implementation("com.google.firebase:firebase-messaging-ktx")
}
```

Click “Sync Now” in the banner that appears at the top of the editor whenever you change a Gradle file.

B.4 Step 4 — Enable Email/Password Authentication

1. In the Firebase console, open Build → Authentication.
2. Click Get started.
3. Under Sign-in method, select Email/Password.
4. Toggle it Enabled and click Save.

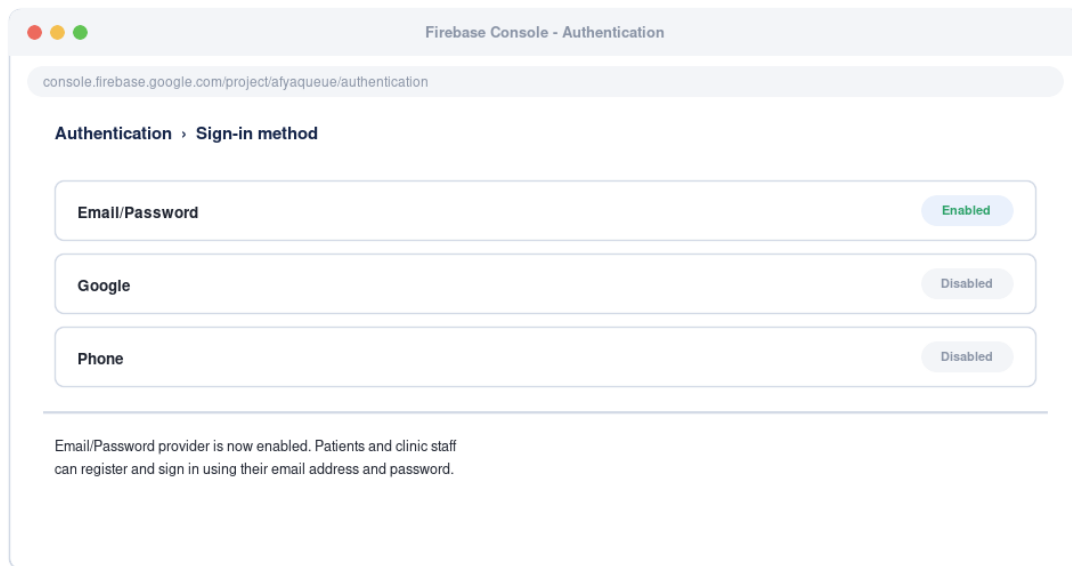


Figure B.3 — Enabling the Email/Password sign-in provider

B.5 Step 5 — Create the Firestore Database

1. In the console, open Build → Firestore Database.
2. Click Create database.
3. Choose Start in test mode for development (we tighten the rules below before publishing).
4. Pick a Cloud Firestore location close to your users (e.g. eur3 or a nearby region) and click Enable.

AfyaQueue uses four top-level collections: users, doctors, appointments and queueStatus — these are created automatically the first time your app writes to them, so there is nothing more to set up manually here.

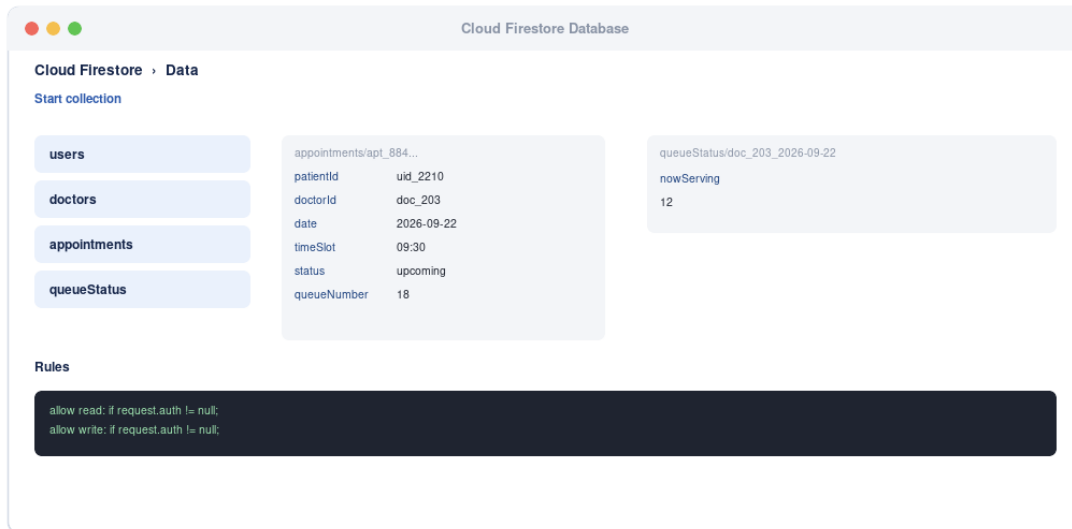


Figure B.4 — The Firestore collections used by AfyaQueue

B.6 Securing your data before launch

Test mode rules allow anyone to read and write your database, which is fine for development but must never ship to production. This is especially important for a healthcare app, where patients' names, contact details and appointment history are sensitive. Before Part F (publishing), replace the rules with something like this:

Firestore Rules

```
// Firestore rules
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }
    match /doctors/{doctorId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null; // tighten to a staff-only check in a real
deployment
    }
    match /appointments/{appointmentId} {
      allow read, write: if request.auth != null;
    }
    match /queueStatus/{docId} {
```

```
allow read: if request.auth != null;  
allow write: if request.auth != null; // called by the staff Call Next Patient screen  
}  
}  
}
```

This requires every read and write to come from a signed-in user — appropriate for a system handling personal health-adjacent information, where (unlike a public listings app) there is no reason to allow anonymous access at all. A production deployment for a real clinic would go further, using Firebase custom claims or a role field lookup to confirm that only accounts with `role == "staff"` can write to `doctors` and `queueStatus`.

Part C — Planning AfyaQueue

“A week of planning saves a month of rewriting.” Before opening the code editor, it pays to be clear about the problem, the architecture, and the screens the app needs. This is exactly the design step a real software team takes between a project brief and the first line of code.

C.1 The problem, recapped

Marion Kibet's original project proposal identifies a real problem faced at healthcare facilities:

- Patients often wait for long, unpredictable periods at hospitals and clinics with no visibility into their position in the queue.
- Appointment booking is frequently done in person or by phone, which is slow and prone to double-booking.
- Reception staff have no simple digital tool to manage the day's queue or notify the next patient.

AfyaQueue's proposed solution is a mobile app that lets patients book appointments with a specific doctor or service, receive a digital queue number, and track their position and estimated wait time in real time, while giving reception staff a simple screen to manage the queue as it moves.

C.2 System architecture

At a high level, AfyaQueue is a thin Android client talking to Firebase over HTTPS — there is no custom server to write or host. The one detail that makes this app different from a simple listings app is that two different users (a patient and a receptionist) must see the same data change in real time, without either one refreshing the screen.

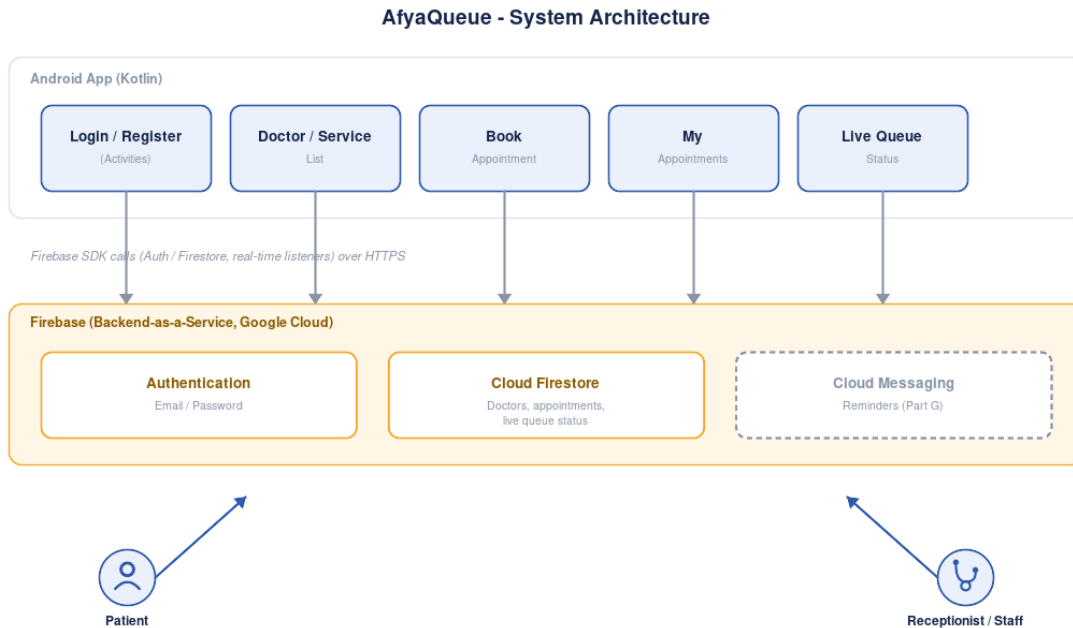


Figure C.1 — AfyaQueue system architecture: Android app, Firebase backend, and real-time queue updates

The app itself is organised around screens that call Firebase SDKs directly: authentication (login/register for both patients and staff), the doctor/service list, appointment booking, the live queue status screen, my appointments, and the staff-facing queue management screens. Firebase handles the “backend” concerns — storing data, authenticating users, and pushing live updates via Firestore's snapshot listeners — so the Android code can focus entirely on user interface and user experience.

C.3 Screens and navigation flow

Sketching the navigation flow before coding avoids a common beginner mistake: building screens in isolation that don't actually connect the way users expect. AfyaQueue also branches early, on the Login screen, into two different experiences — patient and staff — which is worth drawing out explicitly.

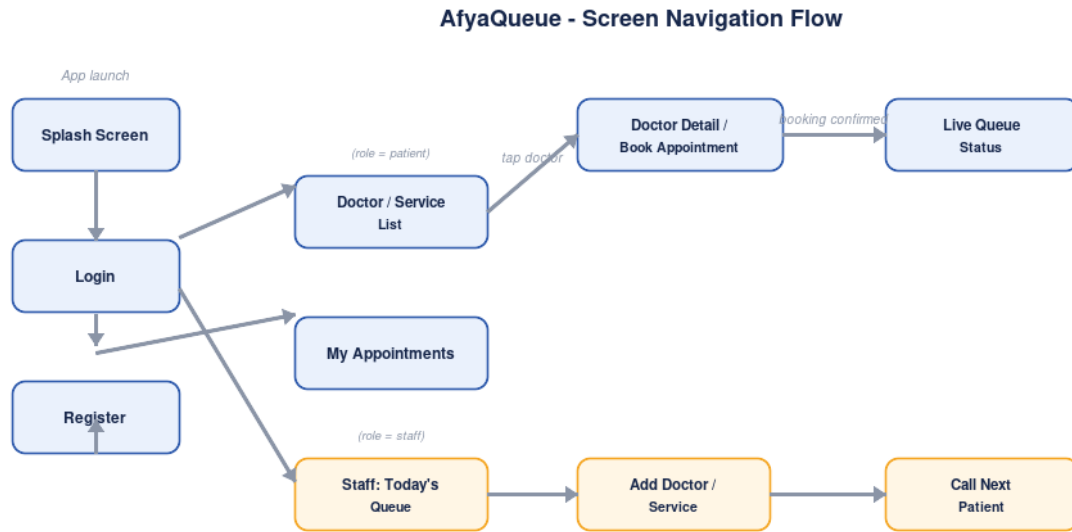


Figure C.2 — AfyaQueue screen navigation flow

Screen	Purpose	Reached from
Splash	Brief branded loading screen while Firebase initialises	App launch
Login	Email/password sign-in for patients and staff	Splash
Register	Create a patient or staff account	Login
Home (Doctor/Service List)	Browse doctors and services as scrollable cards	Login (patient)
Doctor Detail / Book Appointment	Pick a date and time slot, confirm booking	Tapping a card on Home
My Appointments	Upcoming and past bookings, cancel/reschedule	Home (bottom navigation)
Live Queue Status	Patient's queue number, now-serving number, estimated wait	After booking, or from My Appointments

Screen	Purpose	Reached from
Staff Home (Today's Queue)	Reception view of today's queue with Call Next Patient	Login (staff)
Add Doctor / Service	Staff adds a new doctor or service with available time slots	Staff Home

C.4 Data model

AfyaQueue stores its data in four Firestore collections. Keeping the model small and normalised (rather than nesting everything into one giant document) makes queries — like “show today's appointments for Dr. Otieno” — straightforward, and keeps the real-time listener that powers the queue screen fast and cheap.

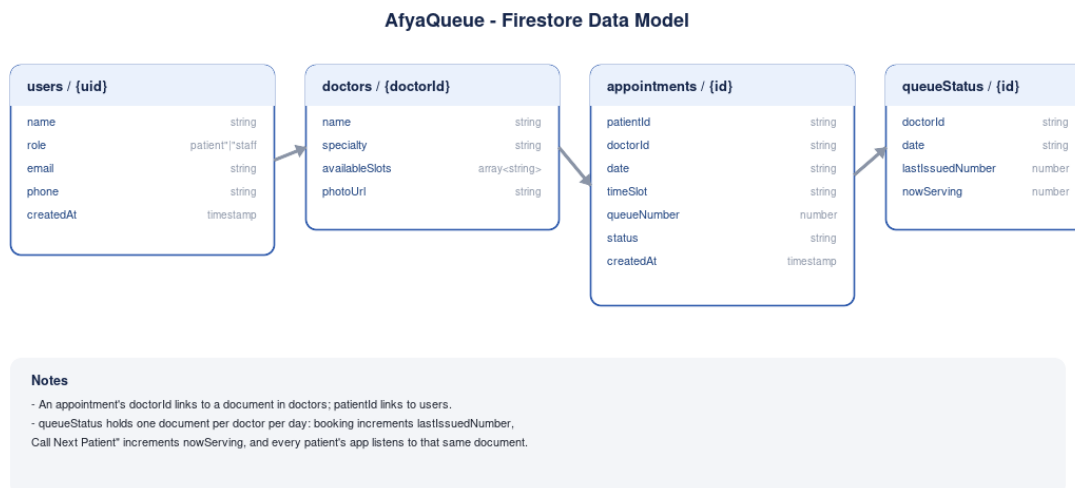


Figure C.3 — Firestore collections used by AfyaQueue and how they relate

Why this shape? An appointment stores a doctorId and patientId rather than embedding the doctor's or patient's full details, so a doctor's profile can be corrected without rewriting every appointment they've ever had. The queueStatus collection holds one small, frequently-updated document per doctor (the number currently being served) — separating this “hot”, fast-changing field from the much larger, rarely-changing appointments collection is what makes the live queue screen update instantly without re-reading every appointment on every change.

C.5 How the live queue actually works

This is the one idea in AfyaQueue that goes a step beyond a typical CRUD (create-read-update-delete) app, so it is worth explaining before writing any code:

1. When a patient books an appointment, the app reads the doctor's current “last issued number” from queueStatus, adds 1, and writes that as the patient's queueNumber on the new appointment document (Part D.4).
2. The patient's Live Queue Status screen attaches a Firestore snapshot listener to that doctor's queueStatus document (Part D.5). A snapshot listener is a subscription: instead of asking the server “has anything changed?” repeatedly, Firestore itself pushes the new data to every listening device the instant it changes.
3. When the receptionist taps Call Next Patient on the Staff Home screen (Part D.6), the app simply updates the nowServing field on that same queueStatus document.
4. That single write is instantly delivered to every patient's phone that is listening — no refresh button, no polling loop, no custom server — which is what makes the queue feel “live”.

Teaching note:

This pattern — one small shared document, updated by one role and observed by another via a snapshot listener — is broadly reusable: the same idea powers chat “typing...” indicators, live sports scores, and ride-hailing driver locations. It is worth pausing on this concept with students before Part D.5.

C.6 Scope for this manual

The original proposal describes a fuller vision including notifications and facility announcements. To keep this manual a realistic, completable classroom project, Part D builds a fully working core covering:

- User registration and login (patient and staff roles)
- Browsable doctor/service list
- Appointment booking with automatic queue-number assignment
- Live queue status with real-time updates
- My Appointments with cancel/reschedule
- Staff Today's Queue with Call Next Patient, and Add Doctor/Service

Two feature areas from the original proposal — push notifications, and a clinic announcements board — are covered as guided extension exercises in Part G, once the core app is working end-to-end. This mirrors how real product teams ship an MVP (minimum viable product) first and layer on enhancements afterwards.

Part D — Building AfyaQueue, Screen by Screen

With the environment installed (Part A), Firebase connected (Part B), and the design agreed (Part C), it's time to build. This part works through every screen in the order a student would naturally build and test them: create the project, add branding, then build outward from login to the doctor list, booking, the live queue screen, and finally the staff-facing screens.

D.0 Create the Android Studio project

1. File → New → New Project → Empty Views Activity → Next.
2. Name: AfyaQueue.
3. Package name: com.moiuniversity.afyaqueue (must match the package you registered in Firebase, Part B.2).
4. Language: Kotlin.
5. Minimum SDK: API 24 (Android 7.0) — covers well over 95% of active devices.
6. Click Finish and wait for Gradle sync to complete.

Then follow Part B.3 to copy in google-services.json and add the Firebase dependencies before continuing.

D.1 App icon and branding

Android apps ship an adaptive icon: a background layer and a foreground layer that the system combines and masks into circles, squircles or other shapes depending on the device. A medical cross reads clearly at small sizes, which makes it a good fit for a healthcare app's launcher icon.

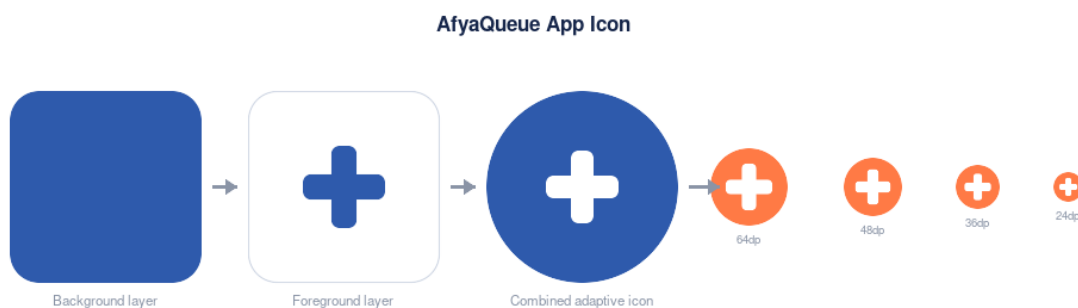


Figure D.1 — AfyaQueue's adaptive app icon: background and foreground layers exported to mipmap sizes

1. Right-click the res folder → New → Image Asset.
2. Icon Type: Adaptive and Legacy.

3. Foreground Layer: choose a simple vector (Clip Art) such as a medical cross glyph, coloured white.
4. Background Layer: a solid colour matching the brand, e.g. #1E7A46 (AfyaQueue green, evoking health and calm).
5. Click Next, then Finish. Android Studio generates every required mipmap size automatically.

Also set the app's display name and brand colours in `res/values/strings.xml` and `res/values/colors.xml`:

res/values/strings.xml

```
<resources>
  <string name="app_name">AfyaQueue</string>
</resources>
```

res/values/colors.xml

```
<resources>
  <color name="brand_green">#1E7A46</color>
  <color name="brand_accent">#C1541A</color>
  <color name="brand_navy">#16264A</color>
</resources>
```

D.2 Data models

Create a new Kotlin file for each data class. These mirror the Firestore documents designed in Part C.4, and Firestore will automatically map documents to and from these classes.

model/User.kt

```
data class User(
  var uid: String = "",
  var name: String = "",
  var role: String = "patient", // "patient" or "staff"
  var email: String = "",
  var phone: String = "",
  var createdAt: Long = System.currentTimeMillis()
)
```

model/Doctor.kt

```
data class Doctor(  
    var id: String = "",  
    var name: String = "",  
    var specialty: String = "", // e.g. "General Practice", "Dentistry"  
    var availableSlots: List<String> = emptyList(), // e.g. ["09:00", "09:30", ...]  
    var photoUrl: String = "",  
    var createdAt: Long = System.currentTimeMillis()  
)
```

model/Appointment.kt

```
data class Appointment(  
    var id: String = "",  
    var patientId: String = "",  
    var patientName: String = "",  
    var doctorId: String = "",  
    var doctorName: String = "",  
    var date: String = "", // "2026-09-18"  
    var timeSlot: String = "", // "09:30"  
    var queueNumber: Int = 0,  
    var status: String = "upcoming", // "upcoming", "completed", "cancelled"  
    var createdAt: Long = System.currentTimeMillis()  
)
```

model/QueueStatus.kt

```
data class QueueStatus(  
    var doctorId: String = "",  
    var date: String = "",  
    var lastIssuedNumber: Int = 0, // incremented each time a new appointment is  
    booked  
    var nowServing: Int = 0 // updated by staff's Call Next Patient action  
)
```

Why a no-argument constructor?

Every field above has a default value, so Kotlin generates a no-argument constructor. Firestore's automatic object mapping (`toObject<Appointment>()`) requires this — without default values, Firestore cannot construct the class when reading a document back.

D.3 Splash, Login and Register

The app opens on a brief splash screen, then either screen depending on whether a user is already signed in. Register additionally lets a new user choose whether they are a Patient or Staff member — this role decides which home screen they land on after logging in.

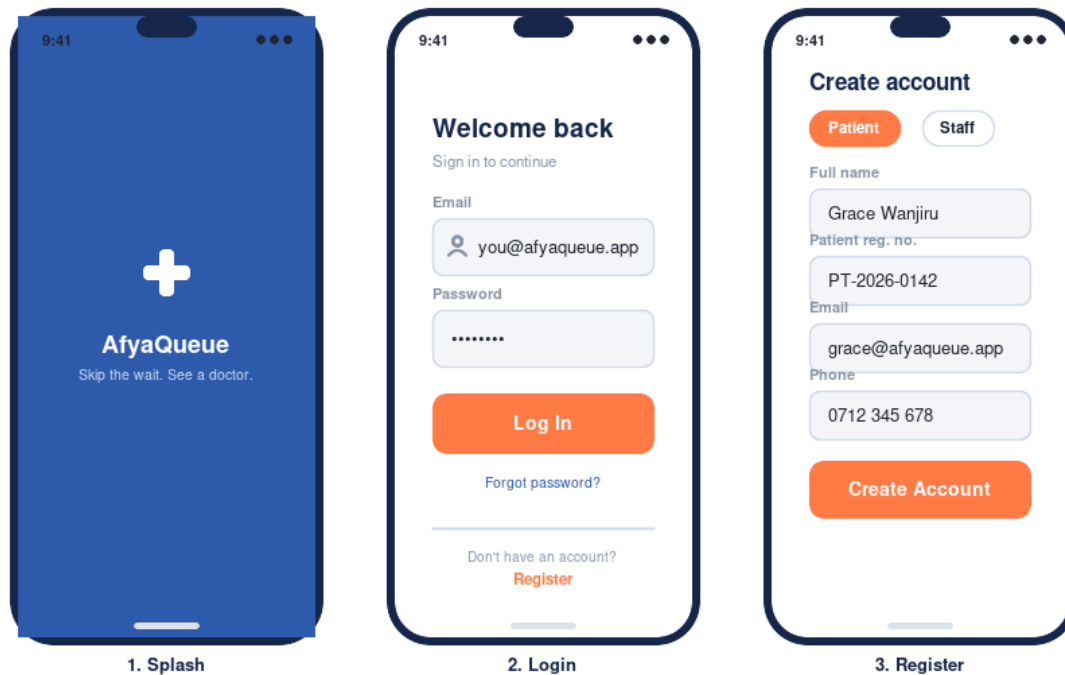


Figure D.2 — Splash, Login, and Register screens

SplashActivity.kt

SplashActivity.kt

```
class SplashActivity : AppCompatActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_splash)

        Handler(Looper.getMainLooper()).postDelayed({
            val user = FirebaseAuth.getInstance().currentUser
            if (user == null) {
                startActivity(Intent(this, LoginActivity::class.java))
                finish()
                return@postDelayed
            }
        })
    }
}
```

```

        // Look up the signed-in user's role before deciding which home screen to open
        FirebaseFirestore.getInstance().collection("users").document(user.uid).get()
            .addOnSuccessListener { doc ->
                val role = doc.getString("role") ?: "patient"
                val destination = if (role == "staff") StaffHomeActivity::class.java else
HomeActivity::class.java
                startActivity(Intent(this, destination))
                finish()
            }
        }, 1200)
    }
}

```

activity_splash.xml is a simple ConstraintLayout with a green background, the medical-cross logo (ImageView), and two TextViews (“AfyaQueue” and “Skip the wait, know your turn”) centred on screen — matching Figure D.2, screen 1.

LoginActivity.kt

activity_login.xml holds two TextInputLayout fields (email, password), a Log In button, and a “Register” text link. Login itself is identical for patients and staff — the role lookup that routes to the right home screen happens once, in SplashActivity above, and again here after a successful sign-in:

LoginActivity.kt

```

class LoginActivity : AppCompatActivity() {
    private lateinit var auth: FirebaseAuth
    private val db = FirebaseFirestore.getInstance()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_login)
        auth = FirebaseAuth.getInstance()

        val etEmail = findViewById<EditText>(R.id.etEmail)
        val etPassword = findViewById<EditText>(R.id.etPassword)

        findViewById<Button>(R.id.btnLogin).setOnClickListener {
            val email = etEmail.text.toString().trim()

```

```

        val password = etPassword.text.toString().trim()
        if (email.isEmpty() || password.isEmpty()) {
            Toast.makeText(this, "Enter your email and password",
Toast.LENGTH_SHORT).show()
            return@setOnClickListener
        }
        auth.signInWithEmailAndPassword(email, password)
            .addOnSuccessListener { result ->
                val uid = result.user?.uid ?: return@addOnSuccessListener
                db.collection("users").document(uid).get()
                    .addOnSuccessListener { doc ->
                        val role = doc.getString("role") ?: "patient"
                        val destination = if (role == "staff") StaffHomeActivity::class.java else
HomeActivity::class.java
                        startActivity(Intent(this, destination))
                        finish()
                    }
                }
            .addOnFailureListener { e ->
                Toast.makeText(this, "Login failed: ${e.message}",
Toast.LENGTH_LONG).show()
            }
        }

        findViewById<TextView>(R.id.tvRegister).setOnClickListener {
            startActivity(Intent(this, RegisterActivity::class.java))
        }
    }
}

```

RegisterActivity.kt

The register screen (Figure D.2, screen 3) lets a user choose the Patient or Staff role via two toggle chips. In a real clinic deployment, staff accounts would normally be created by an administrator rather than self-registered — self-registration for both roles is used here to keep the manual's core scope simple to test.

RegisterActivity.kt

```

class RegisterActivity : AppCompatActivity() {
    private lateinit var auth: FirebaseAuth
    private val db = FirebaseFirestore.getInstance()
    private var selectedRole = "patient"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_register)
        auth = FirebaseAuth.getInstance()

        findViewById<Chip>(R.id.chipPatient).setOnClickListener { selectedRole =
"patient" }
        findViewById<Chip>(R.id.chipStaff).setOnClickListener { selectedRole = "staff" }

        findViewById<Button>(R.id.btnCreateAccount).setOnClickListener {
            val name = findViewById<EditText>(R.id.etName).text.toString().trim()
            val email = findViewById<EditText>(R.id.etEmail).text.toString().trim()
            val password = findViewById<EditText>(R.id.etPassword).text.toString().trim()
            val phone = findViewById<EditText>(R.id.etPhone).text.toString().trim()

            if (name.isEmpty() || email.isEmpty() || password.length < 6) {
                Toast.makeText(this, "Fill all fields (password needs 6+ characters)",
Toast.LENGTH_SHORT).show()
                return@setOnClickListener
            }

            auth.createUserWithEmailAndPassword(email, password)
                .addOnSuccessListener { result ->
                    val uid = result.user?.uid?: return@addOnSuccessListener
                    val user = User(uid = uid, name = name, role = selectedRole, email = email,
phone = phone)
                    db.collection("users").document(uid).set(user)
                    .addOnSuccessListener {
                        val destination = if (selectedRole == "staff")
StaffHomeActivity::class.java else HomeActivity::class.java
                        startActivity(Intent(this, destination))
                        finish()
                    }
                }
        }
    }
}

```

```
    }  
    .addOnFailureListener { e ->  
        Toast.makeText(this, "Registration failed: ${e.message}",  
        Toast.LENGTH_LONG).show()  
    }  
}  
}  
}
```

Try it:

Run the app now (Part A.6 showed you how). Register one patient account and one staff account, then log out and back in as each — confirm each role lands on a different home screen.

D.4 Home (Doctor / Service List)

The home screen is a patient's landing screen after login: a search bar, specialty filter chips, and a scrollable list of doctor/service cards.

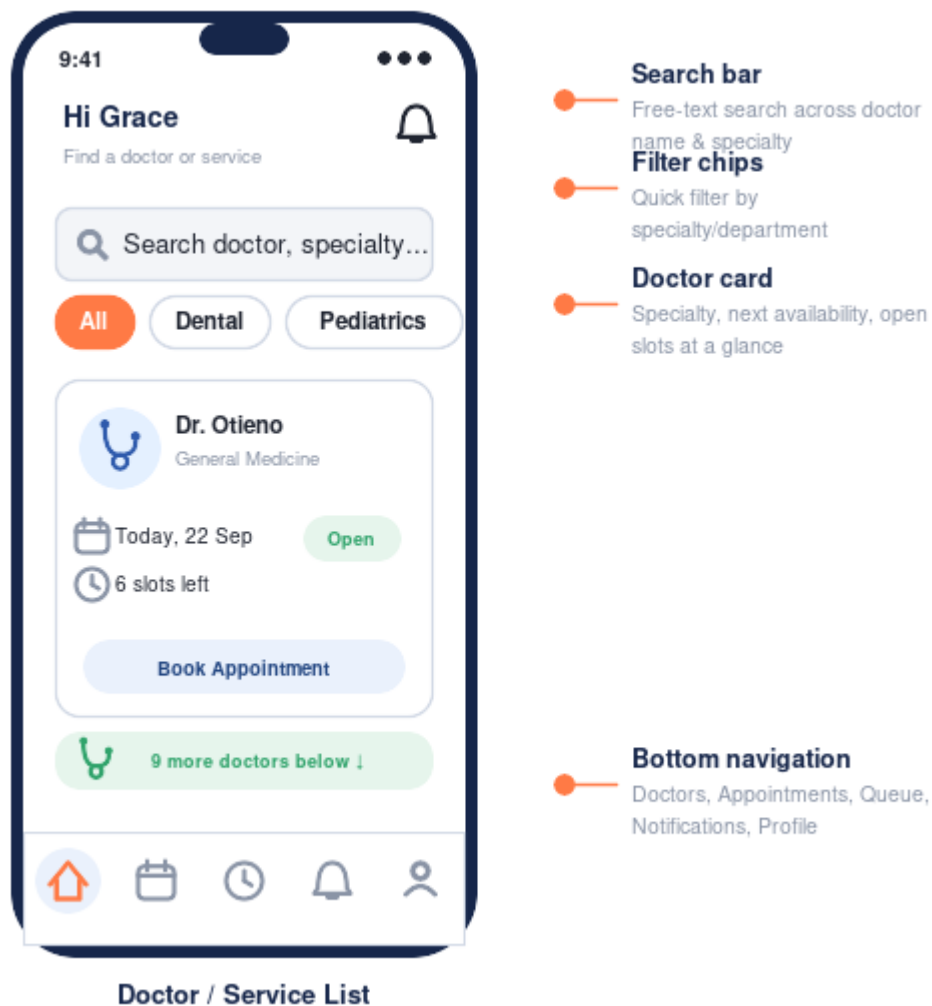


Figure D.3 — Home: search, filter chips and doctor cards

Doctor card layout (item_doctor.xml)

Each card in the RecyclerView shows the doctor's photo, name, specialty and a Book button — use a CardView as the root with an ImageView, two TextViews and a small Button, matching Figure D.3.

DoctorAdapter.kt

```
adapter/DoctorAdapter.kt
```

```

class DoctorAdapter(
    private val doctors: MutableList<Doctor>,
    private val onClick: (Doctor) -> Unit
) : RecyclerView.Adapter<DoctorAdapter.ViewHolder>() {

    class ViewHolder(view: View) : RecyclerView.ViewHolder(view) {
        val image: ImageView = view.findViewById(R.id.imgDoctor)
        val name: TextView = view.findViewById(R.id.tvName)
        val specialty: TextView = view.findViewById(R.id.tvSpecialty)
        val bookBtn: Button = view.findViewById(R.id.btnBook)
    }

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int): ViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_doctor, parent, false)
        return ViewHolder(view)
    }

    override fun onBindViewHolder(holder: ViewHolder, position: Int) {
        val doctor = doctors[position]
        holder.name.text = doctor.name
        holder.specialty.text = doctor.specialty
        Glide.with(holder.image.context)
            .load(doctor.photoUrl)
            .placeholder(R.drawable.placeholder_doctor)
            .circleCrop()
            .into(holder.image)
        val open = { onClick(doctor) }
        holder.itemView.setOnClickListener { open() }
        holder.bookBtn.setOnClickListener { open() }
    }

    override fun getItemCount() = doctors.size

    fun submitList(newList: List<Doctor>) {
        doctors.clear()
        doctors.addAll(newList)
        notifyDataSetChanged()
    }

```

```
}  
}
```

Note on Glide:

Glide (a popular image-loading library) is used here purely for its `circleCrop()` convenience and placeholder handling of doctor photos. It is an optional dependency — add `implementation("com.github.bumptech.glide:glide:4.16.0")` to `build.gradle.kts`, or swap it for a plain `drawable` if you'd rather skip photo uploads entirely for the classroom build.

HomeActivity.kt

HomeActivity.kt

```
class HomeActivity : AppCompatActivity() {  
    private val db = FirebaseFirestore.getInstance()  
    private lateinit var adapter: DoctorAdapter  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_home)  
  
        val recyclerView = findViewById<RecyclerView>(R.id.recyclerDoctors)  
        recyclerView.layoutManager = LinearLayoutManager(this)  
        adapter = DoctorAdapter(mutableListOf()) { doctor ->  
            val intent = Intent(this, DoctorDetailActivity::class.java)  
            intent.putExtra("doctorId", doctor.id)  
            startActivity(intent)  
        }  
        recyclerView.adapter = adapter  
  
        loadDoctors()  
        setupBottomNav()  
    }  
  
    private fun loadDoctors() {  
        db.collection("doctors")  
            .orderBy("name")  
            .get()  
    }  
}
```

```

        .addOnSuccessListener { snapshot ->
            val doctors = snapshot.documents.mapNotNull { doc ->
                doc.toObject(Doctor::class.java)?.apply { id = doc.id }
            }
            adapter.submitList(doctors)
        }
        .addOnFailureListener { e ->
            Toast.makeText(this, "Could not load doctors: ${e.message}",
                Toast.LENGTH_LONG).show()
        }
    }

    private fun setupBottomNav() {
        val nav = findViewById<BottomNavigationView>(R.id.bottomNav)
        nav.setOnItemSelectedListener { item ->
            when (item.itemId) {
                R.id.nav_home -> true
                R.id.nav_appointments -> { startActivity(Intent(this,
                    MyAppointmentsActivity::class.java)); true }
                R.id.nav_profile -> { true /* left as an exercise */ }
                else -> false
            }
        }
    }
}

```

D.5 Doctor Detail / Book Appointment

Tapping a card opens the doctor's detail screen: a date picker row, a grid of that doctor's available time slots, and a Confirm Booking button. This is the screen where a queue number is first assigned — the single most important piece of logic in the whole app.

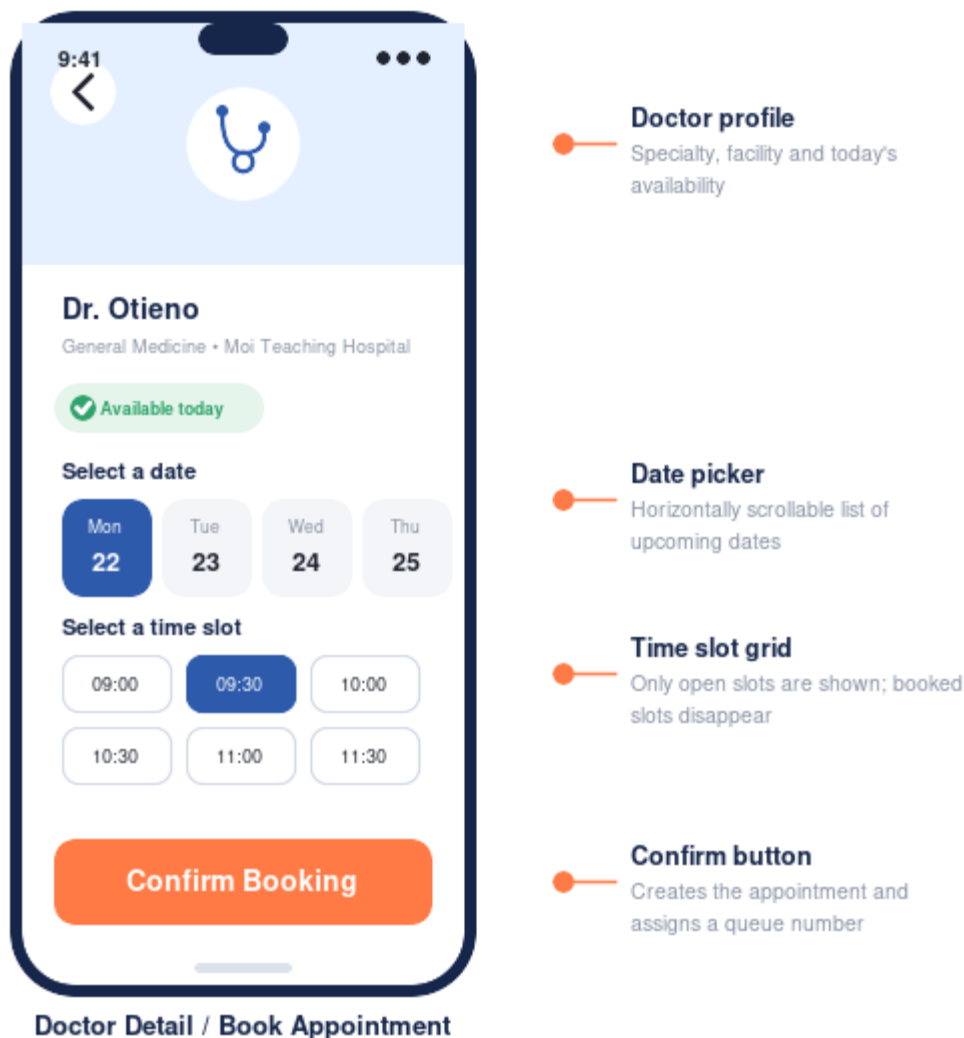


Figure D.4 — Doctor Detail / Book Appointment, with date and time-slot selection

How the queue number is assigned: booking reads the doctor's queueStatus document for the chosen date, adds 1 to lastIssuedNumber, and writes both the updated queueStatus and the new appointment together. Firestore's runTransaction is used so that two patients booking at the exact same moment can never be handed the same queue number — the transaction retries automatically if it detects a conflicting write.

DoctorDetailActivity.kt

```
class DoctorDetailActivity : AppCompatActivity() {
    private val db = FirebaseFirestore.getInstance()
    private lateinit var doctor: Doctor
    private var selectedDate = todayString() // e.g. "2026-09-18"
```

```

private var selectedSlot: String? = null

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_doctor_detail)
    val doctorId = intent.getStringExtra("doctorId") ?: return finish()

    db.collection("doctors").document(doctorId).get()
        .addOnSuccessListener { doc ->
            doctor = doc.toObject(Doctor::class.java)?.apply { id = doc.id } ?:
return@addOnSuccessListener
            bindDoctor()
        }

    findViewById<Button>(R.id.btnConfirmBooking).setOnClickListener {
bookAppointment() }
    }

private fun bindDoctor() {
    findViewById<TextView>(R.id.tvName).text = doctor.name
    findViewById<TextView>(R.id.tvSpecialty).text = doctor.specialty
    val slotGroup = findViewById<ChipGroup>(R.id.chipGroupSlots)
    for (slot in doctor.availableSlots) {
        val chip = Chip(this).apply {
            text = slot
            isCheckable = true
            setOnClickListener { selectedSlot = slot }
        }
        slotGroup.addView(chip)
    }
}

private fun bookAppointment() {
    val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return
    val slot = selectedSlot
    if (slot == null) {
        Toast.makeText(this, "Pick a time slot first", Toast.LENGTH_SHORT).show()
        return
    }
}

```

```

    }

    val queueStatusRef =
db.collection("queueStatus").document("${doctor.id}_${selectedDate}")
    val appointmentRef = db.collection("appointments").document()

    db.runTransaction { transaction ->
        val snapshot = transaction.get(queueStatusRef)
        val currentLast = snapshot.getLong("lastIssuedNumber")?.toInt() ?: 0
        val nextNumber = currentLast + 1

        transaction.set(
            queueStatusRef,
            QueueStatus(doctorId = doctor.id, date = selectedDate, lastIssuedNumber =
nextNumber,
                nowServing = snapshot.getLong("nowServing")?.toInt() ?: 0)
        )
        transaction.set(
            appointmentRef,
            Appointment(
                id = appointmentRef.id, patientId = uid,
                patientName = FirebaseAuth.getInstance().currentUser?.displayName ?: "",
                doctorId = doctor.id, doctorName = doctor.name,
                date = selectedDate, timeSlot = slot, queueNumber = nextNumber
            )
        )
        nextNumber
    }.addOnSuccessListener { queueNumber ->
        val intent = Intent(this, QueueStatusActivity::class.java)
        intent.putExtra("doctorId", doctor.id)
        intent.putExtra("date", selectedDate)
        intent.putExtra("queueNumber", queueNumber as Int)
        startActivity(intent)
        finish()
    }.addOnFailureListener { e ->
        Toast.makeText(this, "Booking failed: ${e.message}",
Toast.LENGTH_LONG).show()
    }
}

```

```
}  
  
private fun todayString(): String {  
    val fmt = java.text.SimpleDateFormat("yyyy-MM-dd",  
java.util.Locale.getDefault())  
    return fmt.format(java.util.Date())  
}  
}
```

Why a transaction, not a plain read-then-write?

Without a transaction, two patients tapping Confirm Booking within the same second could both read `lastIssuedNumber = 12`, and both write back 13 — two patients would collide on the same queue number. `runTransaction` re-reads and retries automatically if another write slipped in first, guaranteeing every number is issued exactly once.

D.6 My Appointments

My Appointments lists a patient's own bookings in two tabs — Upcoming and Past — with a Cancel action on upcoming ones.

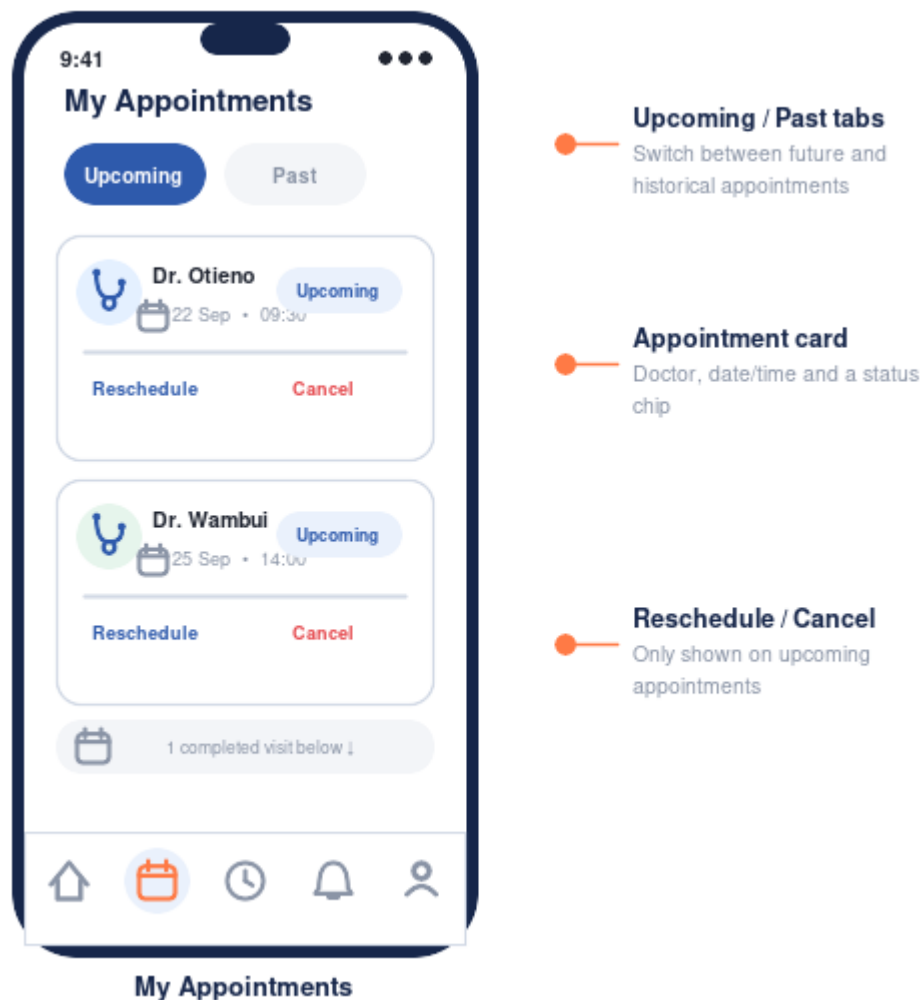


Figure D.5 — *My Appointments: Upcoming and Past tabs*

MyAppointmentsActivity.kt

```
class MyAppointmentsActivity : AppCompatActivity() {
    private val db = FirebaseFirestore.getInstance()
    private lateinit var adapter: AppointmentAdapter

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_my_appointments)
    }
}
```

```

val recyclerView = findViewById<RecyclerView>(R.id.recyclerAppointments)
recyclerView.layoutManager = LinearLayoutManager(this)
adapter = AppointmentAdapter(mutableListOf()) { appointment ->
    val intent = Intent(this, QueueStatusActivity::class.java)
    intent.putExtra("doctorId", appointment.doctorId)
    intent.putExtra("date", appointment.date)
    intent.putExtra("queueNumber", appointment.queueNumber)
    startActivity(intent)
}
recyclerView.adapter = adapter

findViewById<TabLayout>(R.id.tabLayout).addOnTabSelectedListener(object :
TabLayout.OnTabSelectedListener {
    override fun onTabSelected(tab: TabLayout.Tab) {
        loadAppointments(status = if (tab.position == 0) "upcoming" else
"completed")
    }
    override fun onTabUnselected(tab: TabLayout.Tab) {}
    override fun onTabReselected(tab: TabLayout.Tab) {}
})
loadAppointments(status = "upcoming")
}

private fun loadAppointments(status: String) {
    val uid = FirebaseAuth.getInstance().currentUser?.uid ?: return
    db.collection("appointments")
        .whereEqualTo("patientId", uid)
        .whereEqualTo("status", status)
        .orderBy("createdAt", Query.Direction.DESENDING)
        .get()
        .addOnSuccessListener { snapshot ->
            val appointments = snapshot.documents.mapNotNull {
it.toObject(Appointment::class.java) }
            adapter.submitList(appointments)
        }
}
}

```

```
fun cancelAppointment(appointment: Appointment) {  
    db.collection("appointments").document(appointment.id)  
        .update("status", "cancelled")  
        .addOnSuccessListener { loadAppointments(status = "upcoming") }  
}  
}
```

AppointmentAdapter.kt follows the same RecyclerView.Adapter pattern as DoctorAdapter above: each row (appt_card in Figure D.5) shows the doctor's name, date/time, queue number, and status pill, with a Cancel button visible only on the Upcoming tab.

D.7 Live Queue Status — the heart of AfyaQueue

This screen is what makes AfyaQueue more than a booking form: it shows the patient's queue number, the number currently being served, how many people are ahead, and an estimated wait — and every one of those numbers updates on its own, the instant a receptionist calls the next patient, with no refresh button.

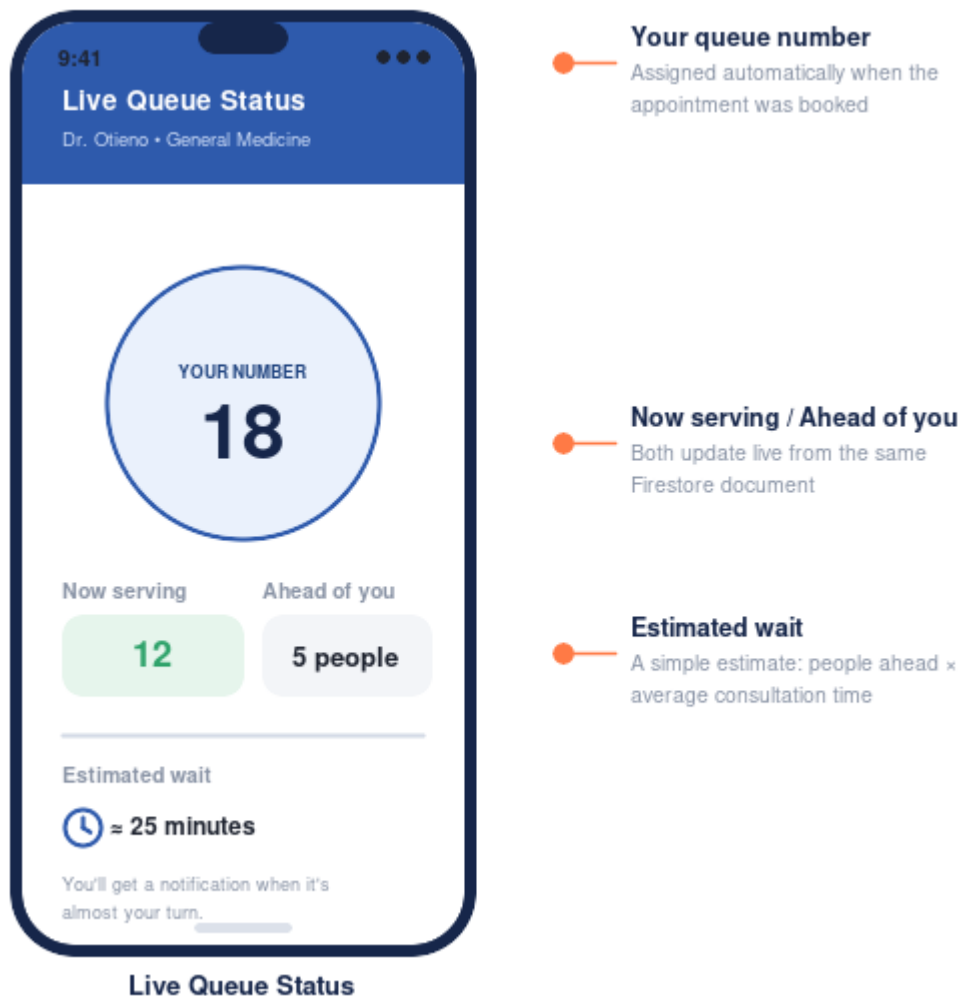


Figure D.6 — Live Queue Status: queue number, now serving, and estimated wait

QueueStatusActivity.kt

```
class QueueStatusActivity : AppCompatActivity() {  
    private val db = FirebaseFirestore.getInstance()  
    private var listenerRegistration: ListenerRegistration? = null  
    private var myQueueNumber = 0
```

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_queue_status)

    val doctorId = intent.getStringExtra("doctorId") ?: return finish()
    val date = intent.getStringExtra("date") ?: return finish()
    myQueueNumber = intent.getIntExtra("queueNumber", 0)
    findViewById<TextView>(R.id.tvMyNumber).text = myQueueNumber.toString()

    // A snapshot listener subscribes to this one small document. Firestore pushes
    every
    // update to this device automatically — there is no polling loop to write.
    val queueStatusRef = db.collection("queueStatus").document("${doctorId}_${date}")
    listenerRegistration = queueStatusRef.addSnapshotListener { snapshot, error ->
        if (error != null || snapshot == null || !snapshot.exists())
    return@addSnapshotListener
        val nowServing = snapshot.getLong("nowServing")?.toInt() ?: 0
        updateUi(nowServing)
    }
}

private fun updateUi(nowServing: Int) {
    val ahead = (myQueueNumber - nowServing - 1).coerceAtLeast(0)
    findViewById<TextView>(R.id.tvNowServing).text = "Now serving:
$nowServing"
    findViewById<TextView>(R.id.tvAhead).text = "Ahead of you: $ahead people"
    // A simple estimate: assume ~5 minutes per patient. A real deployment would
    refine
    // this using each doctor's actual historical average consultation time.
    findViewById<TextView>(R.id.tvEta).text = "≈ ${ahead * 5} minutes"

    if (myQueueNumber <= nowServing) {
        findViewById<TextView>(R.id.tvYourTurn).apply {
            visibility = View.VISIBLE
            text = "It's your turn — please proceed to the doctor's office"
        }
    }
}

```

```
override fun onDestroy() {  
    super.onDestroy()  
    // Always detach the listener when the screen closes, or it keeps running (and  
    // costing reads) in the background even after the user has navigated away.  
    listenerRegistration?.remove()  
}  
}
```

Try it with two devices:

Log in as a patient on one device (or emulator) and as staff on another. Book an appointment on the patient device, then use the staff device's Call Next Patient button (Part D.8) — watch the patient's screen update within a second, with no action taken on that device at all. This is the single best demo moment in the whole manual.

D.8 Staff Home — Today's Queue

Reception staff see a different home screen entirely: a live list of today's appointments across all doctors, summary cards for Serving/Waiting counts, and a Call Next Patient button that advances the queue.

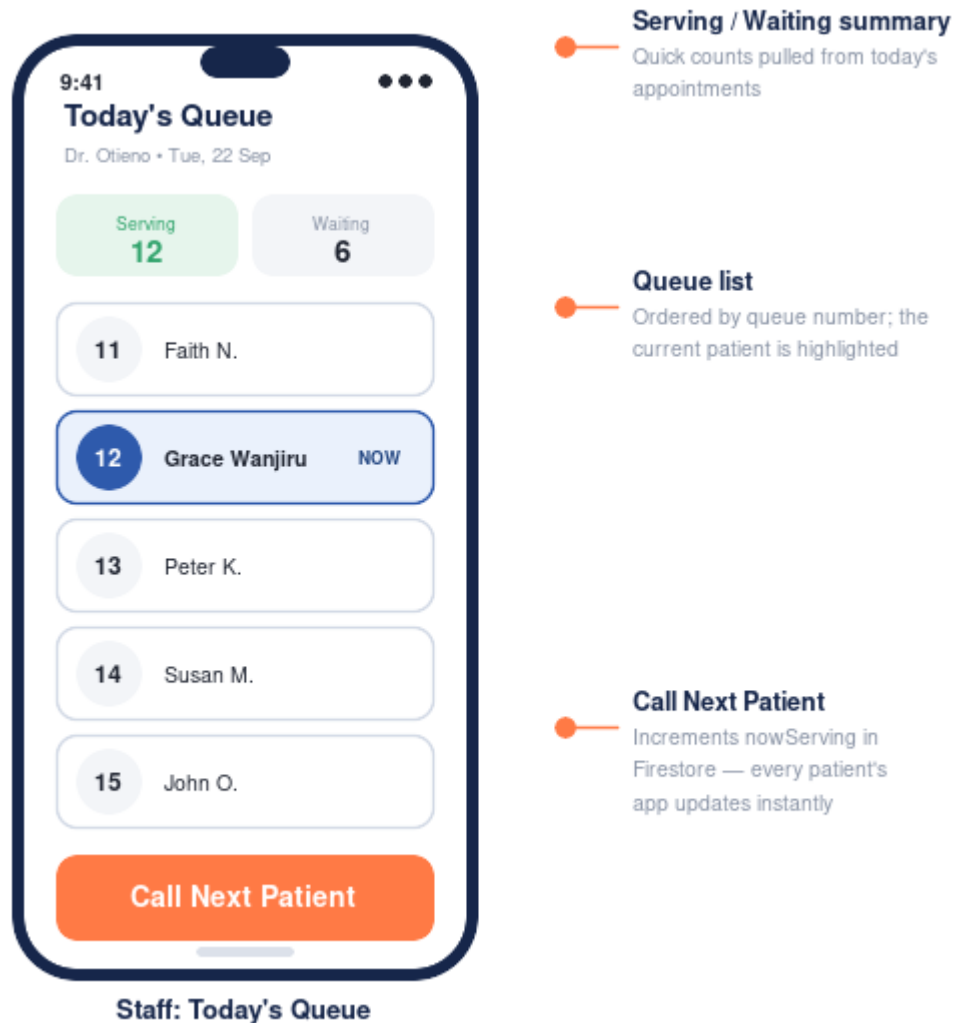


Figure D.7 — Staff Home: Today's Queue with Call Next Patient

StaffHomeActivity.kt

```
class StaffHomeActivity : AppCompatActivity() {  
    private val db = FirebaseFirestore.getInstance()  
    private lateinit var adapter: QueueRowAdapter  
    private var selectedDoctorId: String? = null  
    private val today = todayString()  
}
```

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_staff_home)

    val recyclerView = findViewById<RecyclerView>(R.id.recyclerQueue)
    recyclerView.layoutManager = LinearLayoutManager(this)
    adapter = QueueRowAdapter(mutableListOf())
    recyclerView.adapter = adapter

    findViewById<Button>(R.id.btnCallNext).setOnClickListener { callNextPatient()
}
    findViewById<View>(R.id.btnAddDoctor).setOnClickListener {
        startActivity(Intent(this, AddDoctorActivity::class.java))
    }

    loadDoctorTabs()
}

private fun loadDoctorTabs() {
    db.collection("doctors").get().addOnSuccessListener { snapshot ->
        val doctors = snapshot.documents.mapNotNull {
it.toObject(Doctor::class.java)?.apply { id = it.id } }
        // Populate a doctor-selector spinner/tabs here; selecting one calls
loadQueue(doctorId).
        doctors.firstOrNull()?.let { loadQueue(it.id) }
    }
}

private fun loadQueue(doctorId: String) {
    selectedDoctorId = doctorId
    db.collection("appointments")
        .whereEqualTo("doctorId", doctorId)
        .whereEqualTo("date", today)
        .whereEqualTo("status", "upcoming")
        .orderBy("queueNumber")
        .addSnapshotListener { snapshot, _ ->

```

```

        val appointments = snapshot?.documents?.mapNotNull {
it.toObject(Appointment::class.java) } ?: emptyList()
        adapter.submitList(appointments)
    }

    db.collection("queueStatus").document("${doctorId}_$today")
        .addSnapshotListener { snapshot, _ ->
        val nowServing = snapshot?.getLong("nowServing")?.toInt() ?: 0
        findViewById<TextView>(R.id.tvServingCount).text =
nowServing.toString()
    }
}

private fun callNextPatient() {
    val doctorId = selectedDoctorId ?: return
    val ref = db.collection("queueStatus").document("${doctorId}_$today")
    db.runTransaction { transaction ->
        val snapshot = transaction.get(ref)
        val current = snapshot.getLong("nowServing")?.toInt() ?: 0
        transaction.update(ref, "nowServing", current + 1)
    }.addOnFailureListener { e ->
        Toast.makeText(this, "Could not advance queue: ${e.message}",
Toast.LENGTH_LONG).show()
    }
}

private fun todayString(): String {
    val fmt = java.text.SimpleDateFormat("yyyy-MM-dd",
java.util.Locale.getDefault())
    return fmt.format(java.util.Date())
}
}

```

QueueRowAdapter.kt follows the same RecyclerView.Adapter pattern used throughout the app: each row (queue_row in Figure D.7) shows a patient's queue number, name and time slot, with the row currently being served visually highlighted.

Why addSnapshotListener here too?

Staff Home listens to the same appointments and queueStatus data the patient screens do. This means if two receptionists are working the front desk on separate tablets, both stay in sync automatically — the same real-time pattern serves both directions.

D.9 Add Doctor / Service (staff role)

Staff add a new doctor or service by entering their name, specialty, and toggling which time slots they're available for.



Figure D.8 — Add Doctor / Service screen

AddDoctorActivity.kt

```
class AddDoctorActivity : AppCompatActivity() {  
    private val db = FirebaseFirestore.getInstance()  
  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_add_doctor)  
    }  
}
```

```

        findViewById<Button>(R.id.btnSave).setOnClickListener { saveDoctor() }
    }

    private fun saveDoctor() {
        val name = findViewById<EditText>(R.id.etName).text.toString().trim()
        val specialty = findViewById<EditText>(R.id.etSpecialty).text.toString().trim()
        val slots = collectSelectedSlots()

        if (name.isEmpty() || specialty.isEmpty() || slots.isEmpty()) {
            Toast.makeText(this, "Fill in a name, specialty, and at least one time slot",
                Toast.LENGTH_SHORT).show()
            return
        }

        val doctor = Doctor(name = name, specialty = specialty, availableSlots = slots)
        db.collection("doctors").add(doctor)
            .addOnSuccessListener {
                Toast.makeText(this, "Doctor added", Toast.LENGTH_SHORT).show()
                finish()
            }
            .addOnFailureListener { e ->
                Toast.makeText(this, "Could not save: ${e.message}",
                    Toast.LENGTH_LONG).show()
            }
    }

    private fun collectSelectedSlots(): List<String> {
        val group = findViewById<ChipGroup>(R.id.chipGroupSlots)
        return group.checkedChipIds.map { id -> findViewById<Chip>(id).text.toString() }
    }
}

```

D.10 Wiring it all together

Double-check every Activity is declared in AndroidManifest.xml, and that the internet permission the app needs is requested:

AndroidManifest.xml

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <uses-permission android:name="android.permission.INTERNET" />

  <application
    android:name=".AfyaQueueApp"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/Theme.AfyaQueue">

    <activity android:name=".SplashActivity" android:exported="true">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
    <activity android:name=".LoginActivity" />
    <activity android:name=".RegisterActivity" />
    <activity android:name=".HomeActivity" />
    <activity android:name=".DoctorDetailActivity" />
    <activity android:name=".MyAppointmentsActivity" />
    <activity android:name=".QueueStatusActivity" />
    <activity android:name=".StaffHomeActivity" />
    <activity android:name=".AddDoctorActivity" />
  </application>
</manifest>
```

At this point AfyaQueue is functionally complete: a patient can register, log in, browse doctors, book an appointment and receive a queue number, watch their live queue position update in real time, and manage their appointments; staff can log in, see today's queue, call the next patient, and add new doctors. Part E covers testing this thoroughly — especially the real-time behaviour — before Part F publishes it.

Part E — Testing AfyaQueue

A feature that “works on my machine once” is not the same as a tested feature. This part covers running the app on both an emulator and a real phone, working through a structured test checklist — with special attention to the real-time queue behaviour — and using Logcat to chase down bugs.

E.1 Running on the emulator

With the Pixel 7 (API 34) emulator created in Part A.4 selected in the device dropdown, click the green Run  button. Android Studio builds the app, installs it, and launches it automatically.

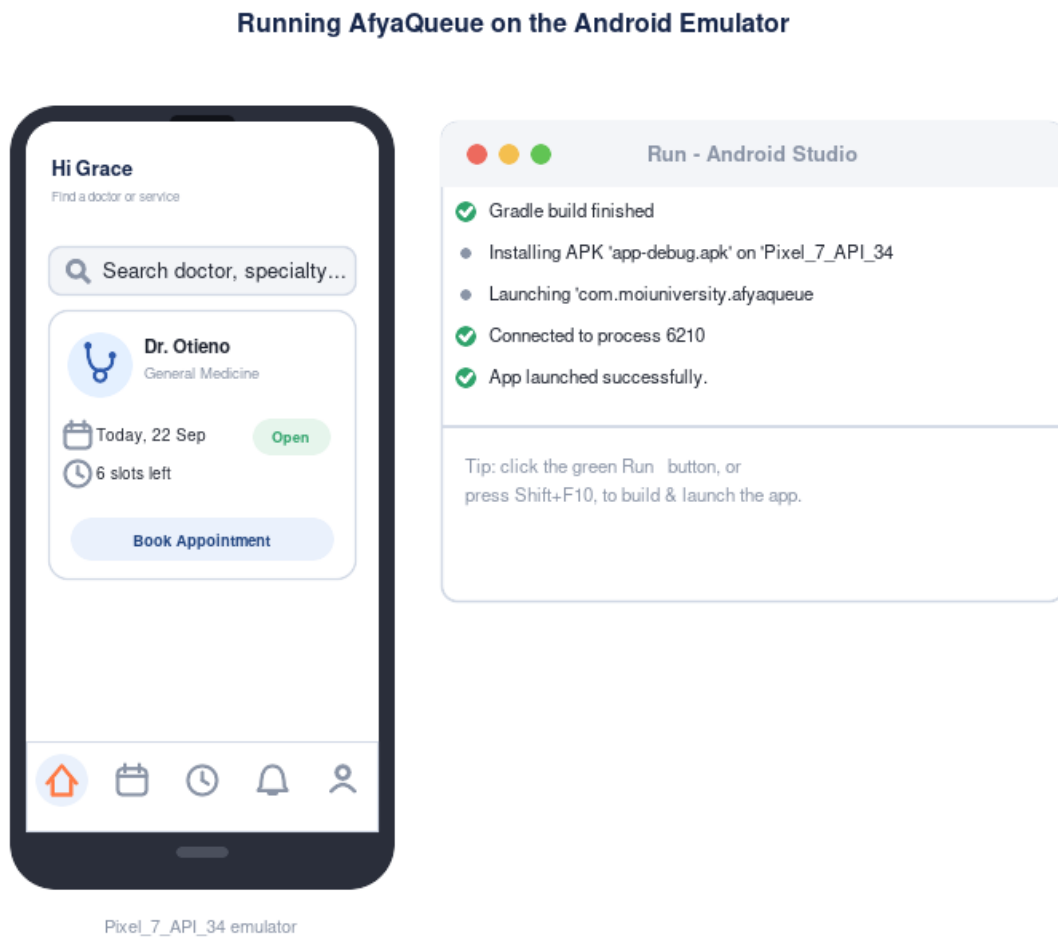


Figure E.1 — AfyaQueue running on the Android Emulator, launched from Android Studio's Run panel

E.2 Running on a physical device

1. On the phone: Settings → About phone → tap Build number seven times to unlock Developer options.
2. Settings → Developer options → enable USB debugging.
3. Connect the phone to the computer with a USB cable and accept the “Allow USB debugging?” prompt on the phone.
4. The device now appears in Android Studio's device dropdown — select it and click Run .

Testing on a real device catches issues an emulator hides: actual network conditions, real screen sizes, and how the app behaves when it is briefly backgrounded (which is common when a patient checks other apps while waiting).

E.3 Testing the real-time queue — the most important test in this manual

Because AfyaQueue's core value is the live queue screen, it deserves a dedicated multi-device test before anything else:

1. Run the app on two emulators at once (Device Manager → launch a second AVD), or one emulator plus one physical device.
2. On Device 1, register/log in as a patient and book an appointment with any doctor.
3. On Device 2, register/log in as staff and open Staff Home for the same doctor.
4. On Device 1, note the assigned queue number and open the Live Queue Status screen.
5. On Device 2, tap Call Next Patient.
6. Confirm Device 1's screen updates — nowServing, ahead-of-you count, and estimated wait — within a second or two, with no action taken on Device 1.

If it doesn't update instantly:

Check that QueueStatusActivity is using addSnapshotListener (not a one-off get()) on the queueStatus document, and that the document id built on both screens matches exactly — “\${doctorId}_\$date” must be built the same way in DoctorDetailActivity, QueueStatusActivity and StaffHomeActivity, or they will be listening to different documents.

E.4 Test checklist

Work through this checklist for every significant change, not just once at the end:

#	Test case	Expected result
1	Register as a patient with a valid email and 6+ character password	Account created, redirected to Home
2	Register again with the same email	Firestore returns an error; a clear message is shown, not a crash
3	Register as staff, log out, log back in	Redirected to Staff Home, not the patient Home screen
4	As staff, add a doctor with 3+ time slots	Doctor appears in Firestore and in the patient Home list
5	As a patient, book an appointment	A queue number is assigned; Live Queue Status opens showing that number
6	Book two appointments back-to-back with the same doctor	Each gets a distinct, sequential queue number — never a duplicate
7	As staff, tap Call Next Patient repeatedly	nowServing increments by exactly 1 each tap, visible instantly on any patient device watching that doctor
8	Open My Appointments, switch Upcoming/Past tabs	Correct appointments appear in each tab
9	Cancel an upcoming appointment	It disappears from Upcoming and its status updates in Firestore
10	Turn off Wi-Fi/mobile data and open the app	A friendly error appears instead of a frozen screen or crash

#	Test case	Expected result
11	Rotate the device / resize the emulator window	Layouts remain usable, no clipped or overlapping content

E.5 Debugging with Logcat

When something doesn't behave as expected, Logcat is the first place to look. Filter by your app's package name and read log lines from top (oldest) to bottom (newest).

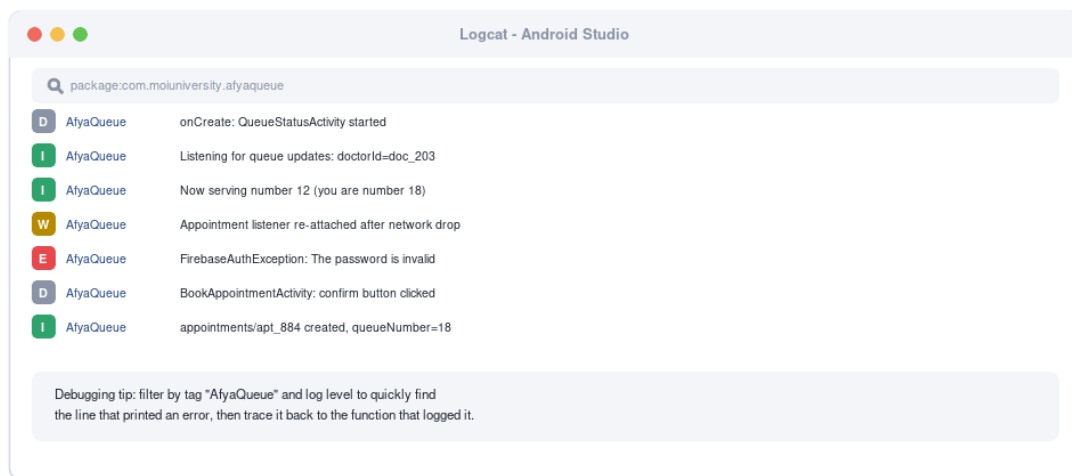


Figure E.2 — Using Logcat to watch app behaviour and catch errors while testing

Add your own log lines while developing to trace what the app is actually doing — this is especially useful around the booking transaction and the snapshot listeners:

Example logging

```
Log.d("AfyaQueue", "bookAppointment: assigned queue number $nextNumber")
Log.d("AfyaQueue", "QueueStatus listener fired: nowServing=$nowServing")
Log.e("AfyaQueue", "Transaction failed", exception)
```

E.6 Common errors and fixes

Symptom	Likely cause	Fix
App crashes immediately on launch	Missing or mismatched google-services.json	Re-download it from Firebase console (Part B.2) and confirm the package name matches exactly
"PERMISSION_DENIED" in Logcat when reading/writing Firestore	Security rules reject the request	Check the user is signed in, and that your rules (Part B.6) allow the operation
Live Queue Status never updates	Listening to a different document id than the one being written	Confirm the "\${doctorId}_\${date}" string is built identically in DoctorDetailActivity, QueueStatusActivity and StaffHomeActivity
Two patients get the same queue number	Booking used a plain read-then-write instead of runTransaction	Re-check Part D.5 — the queue number must be read and incremented inside db.runTransaction
RecyclerView shows nothing	Firestore query returned zero documents, or a missing composite index	Check Logcat for an index-creation link (queries combining whereEqualTo and orderBy need one); confirm test data exists in Firestore

Part F — Publishing to the Google Play Store

Publishing turns a project running on one laptop into an app anyone can install. This part covers preparing release assets, signing the app, and submitting it through Google Play Console.

F.1 Pre-launch checklist

Before generating a release build, gather:

- A 512x512 high-resolution app icon (PNG, no transparency for the Play Store listing icon).
- A 1024x500 feature graphic banner.
- At least 2 phone screenshots (Figures D.3–D.8 in this manual show the kind of screens to capture from your own running app).
- A short description (up to 80 characters) and a full description (up to 4000 characters).
- A privacy policy URL — required because AfyaQueue collects personal data, and in this case health-adjacent data (patient names, contact details, appointment history). A free page on Google Sites or GitHub Pages is sufficient for a class project, but the policy text should be explicit about what health-related data is stored and how it is protected.
- The tightened Firestore security rules from Part B.6 (never ship test-mode rules).

Note for a healthcare-adjacent app:

AfyaQueue as built in this manual stores appointment and contact data but not clinical records (diagnoses, prescriptions, medical history), so it does not require the specialised health-data compliance review some Play Store categories trigger. If a future extension adds clinical data, revisit Play Console's Health apps policy before publishing.

F.2 Generating a signed app bundle

Google Play requires every app to be cryptographically signed. Android Studio generates and manages this signing key for you.

1. Build → Generate Signed Bundle / APK.
2. Choose Android App Bundle (the format Play Store expects) and click Next.
3. Click Create new... to create a keystore the first time. Fill in a keystore password, an alias, an alias password, and certificate details (name, organisation, country).

4. Store the .jks keystore file and its passwords somewhere safe — losing them means you can never update this app again under the same listing.
5. Choose the release build variant and click Finish.

Generate Signed Bundle or APK

Key store path
C:\Users\Student\afyaqueue-release-key.jks [Create new...](#)

Password
.....

Alias
afyaqueue

Alias password
.....

Validity (years)
25

Certificate

First and Last Name	Marion Kibet
Organization	Moi University
Country Code (XX)	KE

Build Variants: [release](#)

[Finish](#)

Figure F.1 — Generating the release keystore via Build > Generate Signed Bundle / APK

Keep this file safe:

Back up your .jks keystore file and its passwords in at least two places (e.g. a password manager and encrypted cloud storage). Google cannot recover a lost signing key, and losing it permanently blocks future updates to the same app listing.

Android Studio produces app-release.aab in app/release/ once signing completes.

F.3 Creating a Google Play Console account

1. Go to play.google.com/console and sign in with a Google account.
2. Pay the one-time USD 25 registration fee (institutions sometimes cover this for student projects — check with your department first).
3. Complete the developer identity verification steps Google requires.

F.4 Creating the app and the store listing

1. In Play Console, click Create app.

2. Enter the app name (AfyaQueue), default language, and confirm it is an App, not a Game, and Free.
3. Accept the Developer Program Policies and US export laws declarations, then click Create app.
4. Open Store presence → Main store listing and fill in the short description, full description, app icon, feature graphic and screenshots gathered in F.1.

The image displays two side-by-side screenshots of the Google Play Console interface. The left screenshot, titled 'Play Console - Create app', shows the 'Create app' form. It includes a URL bar with 'play.google.com/console', a text input for 'App name' containing 'AfyaQueue', a dropdown for 'Default language' set to 'English (United States)', a dropdown for 'App or game' set to 'App', a dropdown for 'Free or paid' set to 'Free', and two checked checkboxes under 'Declarations': 'Developer Program Policies' and 'US export laws'. A green 'Create app' button is at the bottom. The right screenshot, titled 'Play Console - Store listing', shows the 'Main store listing' form. It includes a text input for 'App name' containing 'AfyaQueue', a text input for 'Short description' containing 'Book appointments and track your clinic queue in real time', a text input for 'Full description' containing 'AfyaQueue lets patients book appointments, get a digital queue number...', and three image placeholders for 'App icon', 'Feature graphic', and 'Phone screenshots'. A blue 'Save' button is at the bottom.

Figure F.2 — Creating the app and completing the store listing in Google Play Console

F.5 Content rating, target audience and data safety

Under Policy → App content, complete each required declaration:

- Content rating questionnaire — answer honestly; AfyaQueue has no user-generated public content, which typically simplifies this step.
- Target audience and content — AfyaQueue targets adults (patients and clinic staff), not children.
- Data safety — declare exactly what AfyaQueue collects (name, email, phone number, appointment records) and how it is used and protected, matching the security measures from Part B.6. Be explicit that appointment data is linked to identifiable patients.
- Privacy policy — paste in the URL prepared in F.1.

F.6 Uploading the release

1. Open Release → Testing → Internal testing (recommended first step — lets you and classmates install the app before the public can).

2. Click Create new release.
3. Upload app-release.aab.
4. Add release notes describing what this version does.
5. Click Review release, resolve any warnings Play Console flags, then Start rollout to Internal testing.

Play Console - Release

play.google.com/console/.../release

☒ **Internal testing**
A small group of testers you choose

☐ **Closed testing**
A wider, invite-only group

☐ **Open testing**
Anyone with the link can opt in

☐ **Production**
Everyone on Google Play

app-release.aab

✓ Uploaded • Version 1 (1.0) • 14.1 MB

Release name

1.0 (1)

Release notes

Initial release: book appointments, view live queue status, and manage today's queue.

Review release

Figure F.3 — Choosing a release track and rolling out app-release.aab in Google Play Console

Track	Who can install	Typical use
Internal testing	A small list of testers you add by email	You and your project team / instructor
Closed testing	A larger, invite-only group	Your class, or clinic staff piloting the app
Open testing	Anyone with the opt-in link	Wider beta before public launch
Production	Everyone on Google Play	Public release

F.7 Review and publish timeline

Google typically reviews a new app within a few hours to a few days. Once approved on Internal testing, promote the same release to Closed, then Open, then Production testing tracks as confidence grows — each promotion re-uses the same signed .aab, so there is no need to rebuild.

For a class project:

Internal testing alone is often enough to demonstrate a working, installable app to an instructor and classmates without needing full public review — testers just need to accept an emailed invite link. A live two-device demonstration of the queue updating in real time (Part E.3) makes a strong impression during a project defense.

Part G — Extending AfyaQueue (Stretch Goals)

The original project proposal describes a fuller vision than Part D's core build: push notifications and a facility announcements board. Both are excellent extension exercises — they reuse every pattern already taught (a new Firestore collection or field, a RecyclerView, a Firebase listener) rather than introducing entirely new concepts, which makes them well suited to independent student work or a graded extension task.

G.1 Push Notifications

Design sketch: use Firebase Cloud Messaging (FCM) to notify a patient when their turn is approaching, instead of requiring them to keep the Live Queue Status screen open and watched.

1. The FCM dependency (implementation("com.google.firebase:firebase-messaging-ktx")) was already added in Part B.3, ready for this extension.
2. Create a service extending FirebaseMessagingService and override onMessageReceived() to show a notification.
3. Store each user's FCM token in their users/{uid} document (add a field fcmToken: String) whenever the token refreshes.
4. Trigger a notification either from a small Cloud Function that runs whenever nowServing is updated on a queueStatus document (comparing it to each waiting patient's queueNumber), or (for a simpler class project) manually via the Firebase console's Cloud Messaging → “Send test message” tool while demonstrating the concept.

Scope note:

A full server-triggered notification pipeline normally uses Cloud Functions, which requires the Firebase project to be on the pay-as-you-go Blaze plan (usage still falls within the free tier for a class project, but it does require adding a billing account). Instructors may prefer to treat this as a design/discussion exercise rather than a fully graded build.

G.2 Facility Announcements Board

Design sketch: add an announcements collection so clinic staff can broadcast short notices (e.g. “Dr. Otieno is unavailable this afternoon”) that every patient sees on their Home screen.

Announcements sketch

```
data class Announcement(
```

```

var id: String = "",
var text: String = "",
var postedBy: String = "",
var createdAt: Long = System.currentTimeMillis()
)

// Staff posts an announcement
db.collection("announcements").add(announcement)

// Patients see the latest 5 announcements at the top of Home, live
db.collection("announcements")
    .orderBy("createdAt", Query.Direction.DESENDING)
    .limit(5)
    .addSnapshotListener { snapshot, _ -> /* bind to a small banner list, same listener
pattern as D.7 */ }

```

This reuses exactly the same `addSnapshotListener` pattern taught for the live queue in Part D.7, applied to a different collection — a good way for students to see that the real-time pattern generalises beyond just queue numbers.

G.3 Further improvement ideas

- Doctor ratings from patients after a completed appointment, stored as a subcollection under each doctor's document.
- Reschedule (rather than just cancel) an appointment, re-running the same booking transaction from Part D.5 against a new date/slot.
- Estimated wait time based on a doctor's real historical average consultation time instead of the fixed 5-minutes-per-patient placeholder used in Part D.7.
- Pagination (Firestore's `startAfter()` cursors) once the appointments collection grows beyond a couple of hundred documents per doctor.
- Offline support using Firestore's built-in local cache (enabled by default) so a patient's last known queue position remains visible during a brief connection drop.
- Unit tests for the queue-number transaction logic, and UI tests for the login flow using Espresso.

Appendix

A. Complete Firestore security rules

The production-ready rules introduced in Part B.6, shown here in full for reference:

firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    match /users/{userId} {
      allow read, write: if request.auth != null && request.auth.uid == userId;
    }

    match /doctors/{doctorId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null; // tighten to a staff-only check in a real
deployment
    }

    match /appointments/{appointmentId} {
      allow read, write: if request.auth != null;
    }

    match /queueStatus/{docId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null; // called by the staff Call Next Patient screen
    }

    match /announcements/{announcementId} {
      allow read: if request.auth != null;
      allow write: if request.auth != null; // Part G.2 extension
    }
  }
}
```

A note on "staff-only" checks:

The rules above allow any signed-in user to write to doctors and queueStatus, which is acceptable for a class demonstration but not for a real clinic. A production version would add a role field lookup or Firebase custom claims (e.g. request.auth.token.role == 'staff') so patient accounts cannot modify doctor records or the queue — this is a good discussion point when reviewing this manual with students.

B. Troubleshooting quick reference

Error message	Meaning	Where to look
INSTALL_FAILED_INSUFFICIENT_STORAGE	The emulator or device is out of space	Wipe emulator data (Device Manager → ▼ → Wipe Data) or free device storage
Default FirebaseApp is not initialized	google-services.json missing or plugin not applied	Part B.3 — confirm the file's location and the Gradle plugin lines
PERMISSION_DENIED reading/writing Firestore	Security rules reject the request	Check the user is signed in, and that your rules (Part B.6) allow the operation
FirebaseAuthUserCollisionException	Registering with an email already in use	Show the user a friendly message; offer to log in instead
Unable to resolve host "firestore.googleapis.com"	No internet connection on the device/emulator	Check the emulator's network settings or the device's Wi-Fi/data

Error message	Meaning	Where to look
Live Queue Status screen never updates	Listening to the wrong document, or the listener was never attached	Part E.3 — confirm the <code>"\${doctorId}_\${date}"</code> id matches on both the writer (Staff Home) and reader (Queue Status) screens

C. Glossary

Term	Meaning
Activity	One screen of an Android app
AVD	Android Virtual Device — an emulator configuration
Gradle	The build system Android Studio uses to compile apps and manage dependencies
Firestore	Firebase's cloud NoSQL document database
Snapshot listener	A Firestore subscription that pushes live updates to a device whenever a document changes, without polling
Transaction	A Firestore operation that reads and writes atomically, retrying automatically if another write conflicts — used here to assign unique queue numbers
RecyclerView	An efficient, scrollable list UI component that reuses row views
Intent	An Android object used to start another Activity or request an action from the system

Term	Meaning
APK / AAB	Android Package / Android App Bundle — the installable/distributable formats for an app
Keystore	A file holding the cryptographic key used to sign a release build
Logcat	Android Studio's live log viewer

D. Suggested assessment rubric

For instructors grading this as a practical assignment:

Criterion	Weight	Evidence
Environment set up correctly, app runs on emulator/device	10%	Live demonstration or screen recording
Firebase Authentication (register/login, patient and staff roles) implemented	15%	Code review + live demo
Appointment booking with correct, non-duplicate queue-number assignment	20%	Code review + live demo, including the two-booking test (Part E.4, test 6)
Live queue status updates in real time across devices	20%	Live two-device demonstration (Part E.3)
Staff Today's Queue and Call Next Patient function correctly	10%	Live demo
Code quality: structure, naming, comments	10%	Code review
Testing evidence (checklist from Part E completed)	10%	Submitted checklist / test report

Criterion	Weight	Evidence
Extension feature attempted (Part G)	5%	Code review or design write-up

E. References and further reading

- Android Developers documentation — developer.android.com/docs
- Firebase documentation — firebase.google.com/docs
- Cloud Firestore transactions and batched writes — firebase.google.com/docs/firestore/manage-data/transactions
- Material Design guidelines — m3.material.io
- Google Play Console Help — support.google.com/googleplay/android-developer

End of manual. *Original project concept — AfyaQueue: Healthcare Appointment & Queue Management App — proposed by Marion Kibet (COM/5916/24) for CSC 417. This teaching manual expands that proposal into a full, buildable Android application for classroom instruction.*